



MASLAB

MASLAB 2026 Lecture Notes

MASLAB 2026 Staff
maslab-2026-staff@mit.edu

Last Updated September 27, 2025

Lecture 0

Introduction

Welcome to the MIT Mobile Autonomous Systems Laboratory! We're very excited that you've chosen to learn more about this field, and we can't wait to accompany you on your journey through the wonderful world of autonomous robotics. The MASLAB lecture series is composed of ten lectures that collectively cover the fundamental principles of robotics from both theoretical and practical perspectives. As an important aside, we highly recommend you follow along our lecture slides (accessible on the course website) while reading these notes, as this will help deliver a maximally beneficial learning experience. So without any further ado, let's get started!

0.1 What Is a Robot?

As of this writing, Wikipedia defines a *robot* as “a machine—especially one programmable by a computer—capable of carrying out a complex series of actions automatically.” Robots occupy a critical part of our everyday experience. Industrial robots facilitate modern manufacturing, vacuum robots clean our floors, and humanoid robots are a defining feature in many of our beloved science fiction stories. However, these examples only provide a general idea of what robots can do. To engineer robots and imagine the possibilities they provide to us, we require a robust conceptual model of what a robot really *is*.

In MASLAB, we will use a conceptual model that defines an autonomous robot in terms of three types of fundamental components: one or more *sensors*, a *controller*, and one or more *actuators*. Sensors measure the environment, controllers decide what to do, and actuators can affect the controller's decisions in the environment. Figure 1 shows a diagram representing this conceptual model.

Assume we can represent a system containing the robot and its surrounding environment as $\mathbf{E}$. A robotic sensor S measures the environment and determines a simplified representation of the environment $\mathbf{M}$. We can write this relationship as the equation below, where S represents the sensor. For example, this process could capture the entire environment $\mathbf{E}$ and calculate the position of the robot as $\mathbf{M}$.

$$\mathbf{M} = S(\mathbf{E}) \tag{1}$$

Next, a controller C determines the best action $\mathbf{X}$ from the measured values $\mathbf{M}$. We

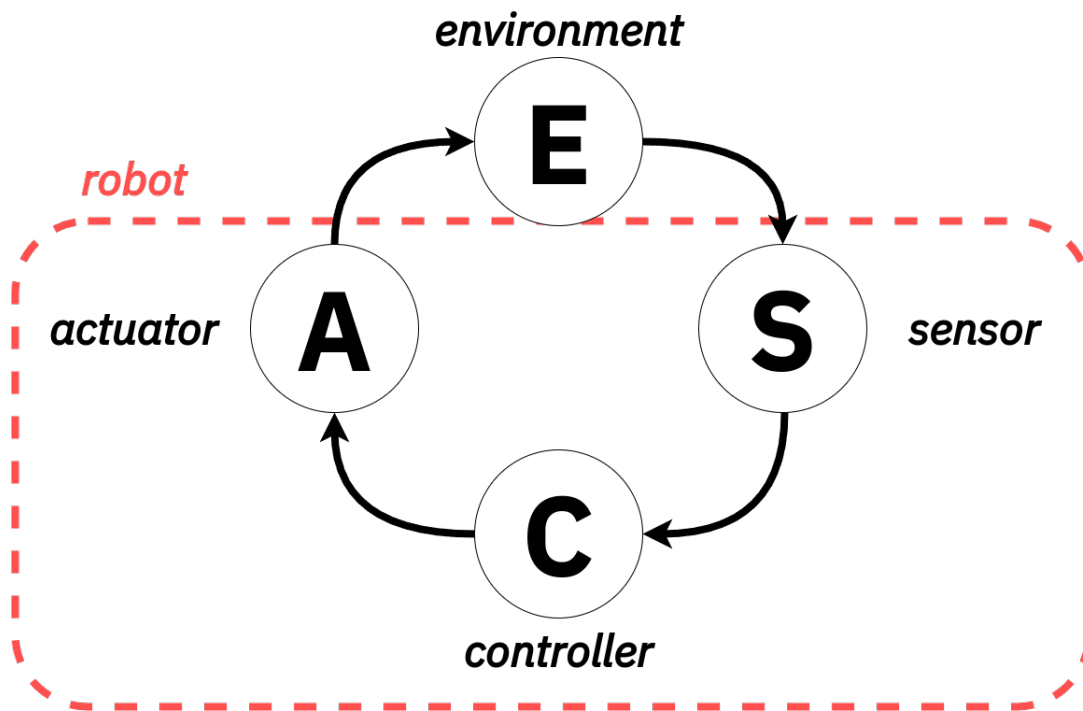


Figure 1: Our Conceptual Model of a Robot

might write this mathematically as follows, where C represents the controller.

$$\mathbf{X} = C(\mathbf{M}) \quad (2)$$

Finally, a robotic actuator A affects the environment. Environment variables (or their time derivatives) are changed in this process, based on the decisions made by the controller. For example, the actuator may affect the acceleration of the robot, which causes change to the robot's position in the environment. We can write this relation below, where A represents the actuator.

$$\mathbf{E} = A(\mathbf{X}) \quad (3)$$

We can see that the three functions S , C , and A provide a cohesive model for a robotic system. Don't worry about the notation here; this is just to get you thinking about the conceptual architecture of robotic systems. The key takeaway here is that robots are collections of *sensors*, *controllers*, and *actuators* that can respond to stimuli in their environment.

0.2 What Will I Learn?

So glad you asked! As mentioned before, the MASLAB lecture series is composed of ten lectures on the fundamentals of robotics. These are the topics of each lecture.

0. **Introduction** – you are here!

1. **Robot Operating System (ROS).** ROS is a software tool used by widely across the robotics industry to provide a framework for controller software. It's extremely helpful, and we're excited to share with you all it has to offer.
2. **Version Control.** You will be developing a lot of software to control your robot well, and you'll end up with lots (and lots) and different versions of your software. This is where version control comes in.
3. **The Kitbot.** The course staff will provide to you a robot kit, from which you'll construct Kitbot. This is intended to jumpstart your robotics adventure, as the versatility of Kitbot opens the door of opportunity wide to you and your team!
4. **Sensors & Odometry.** How does your robot know where it is in space? We'll learn how to process information from sensors and estimate the robot's position in space.
5. **Principles of Design.** Mechanical, electrical, and software engineering are hard. This lecture will offer you some best practices in these domains. Following these practices will help you keep your project organized and robust as it grows.
6. **Control Theory.** Computers always do exactly what you tell them to do, but almost never exactly what you want them to do. On the other hand, robots, which exist in the chaotic and noisy real world, never do *exactly* what you tell them to. Control theory provides us with a rigorous way to think about analyzing sensor input in real-time to determine control efforts for our actuators. This process allows us to correct for non-ideal conditions in the environment or imperfections in our own handiwork.
7. **Kinematics & Path Planning.** How do you get from A to B? We'll teach you the fundamentals of robot kinematics and path planning, which will allow you to intelligently chart your robot's course across the game field and towards success!
8. **Computer Vision.** You won't be able to control your robot during the competition, so it will have to see with its own eyes. Computer vision algorithms allow your robot to identify the locations of walls and game pieces in camera pictures.
9. **Machine Learning.** The field of machine learning is constantly improving and advancing, and it has many treasures to offer the field of autonomous robotics. We'll explore some simple machine learning processes that you can use to supercharge your robots.

0.3 How Is the Course Structured?

The core of the MASLAB educational experience is its hands-on pedagogical model. In coordination with our lecture series, you will have the opportunity to explore the fundamentals of autonomous robotics with the guidance of our course staff. Below is an outline of the course structure.

Lectures We will be offering lectures during the first week of IAP, Monday through Friday, from 10 am to 12 pm. Lectures will be offered on the first floor of the Stata Center in Room 32-141.

Lab Hours Our lab space is located in Room 34-501. Our course staff will be available to guide and coach you through your robotics journey on Mondays through Fridays from 10 am to 5 pm, except during lecture. Additionally, the lab will be closed on Monday, January 19 in honor of Martin Luther King, Jr.

Design Reviews During the second week of IAP, the course staff will meet individually with each team for 30-minute Design Reviews. During this coaching meeting, you will describe to us your preliminary design, and the staff will answer any questions and offer you coaching guidance forward to help you refine and implement your design. **You do not need to have a completed design** to participate in the Design Review, though we strongly encourage bringing any sketches, CAD, or prototypes you may have. Your team will have the opportunity to schedule the meeting at a time that is most convenient for you. We strongly encourage all team members to attend the Design Review, but one team member will be sufficient.

Final Competition The MASLAB Final Competition will be on Thursday, January 29 at 10 am in Room 26-100. We expect your attendance and participation in the Final Competition, though if you are unable to attend or compete, we request that you notify the MASLAB Course Staff in person, through a private Piazza post, or via an email to maslab-2026-staff@mit.edu at your earliest opportunity.

0.4 How Is the Course Graded?

MASLAB represents your individual journey through the novel and exciting field of autonomous robotics. As such, the course is intended for people of all experience levels in the field to challenge themselves in a learning environment supported by the coaching and mentorship of our course staff. This unique pedagogical model drives our grading policy. MASLAB is graded 6 units pass/fail. To earn a P (passing grade) in MASLAB, we ask that you follow these four guidelines.

- Ensure your team participates in a Design Review with the course staff. You are not required to attend the Design Review to complete this requirement, but if you are available, we strongly encourage your attendance.
- Attend the MASLAB Final Competition and, with your team, submit a robot to compete. The performance of your robot will have no influence on your grade. All members of your team should attend the competition, but if you cannot attend, please contact the course staff at your earliest convenience and you will be excused.

- Out of respect for our generous sponsors whose kindness has provided our robotic hardware, treat all lab hardware and tools with respect, and report any damaged hardware to the course staff.
- Out of respect for your fellow students, treat your teammates, other teams, the course staff, and the lab staff with respect.

0.5 Battery Safety

Lithium batteries require proper handling! Your Kitbot runs on a rechargeable lithium-polymer (LiPo) battery, which is the state of the art in chemical energy storage. LiPo batteries have unrivaled energy density and power density... but **with great power comes great responsibility**. Please review this section and its associated lecture slides in detail before working with LiPo batteries in the lab. LiPo batteries are not unsafe when handled correctly, but when handled incorrectly, they can present a serious fire and electrocution hazard.

0.5.1 Battery Ratings

LiPo batteries are typically labeled with three numbers, like so.

5200mAh 3S 80C

Capacity 5200mAh indicates that the capacity of the LiPo battery is 5200 milliamp-hours. This means that the battery, on a full charge, can provide 5.2 amps for one hour. Current and time are inversely related, so doubling the current cuts the lifetime in half and vice versa.

Connectivity 3S indicates that your LiPo battery is composed of three lithium-polymer cells connected in series. Every lithium-polymer cell stores 3.7 volts in its nominal state, so the nominal voltage of a 3S battery is $3.7\text{ V} \cdot 3 = 11.1\text{ V}$.

Max Current 80C indicates the maximum discharge current of the battery. 1C is defined as the discharge current required to drain the battery in one hour. In this case, 1C equals 5.2 amps, and $80\text{C} = 5.2\text{ A} \cdot 80 = 416\text{ A}$. Four hundred amps is certainly enough to melt the 18-gauge wires coming out of the LiPo and almost anything else in the way of this current path. We strongly discourage you from short-circuiting your LiPo battery for this reason.

0.5.2 Cell Voltages and Balancing

Each lithium-polymer cell in your battery should be kept within its acceptable voltage range. The maximum recommended voltage for each cell is 4.2 volts, and the minimum recommended voltage for each cell is 3.0 volts. Furthermore, all three cells should be kept very

close in voltage level as the battery charges and discharges. This is referred to as *cell balancing*. Please **notify the course staff immediately** if any cell in your battery becomes overcharged ($> 4.2\text{ V}$) or undercharged ($< 3.0\text{ V}$), or if your cells become unbalanced.

0.5.3 Don't Short-Circuit the Battery

Please.

0.5.4 Safety Precautions

Throughout the entirety of the MASLAB course, inside or outside the lab, please always follow these safety precautions. By doing so, you will show respect towards yourself, your teammates, and your peers, whose safety these precautions are meant to protect.

- Never short-circuit the battery.
- Always store the battery in the provided fireproof bag when not in use.
- Always connect the provided voltage monitor when using the battery.
- Never charge your battery unattended or overnight.
- Report any puffy, overheated, overcharged, undercharged, or damaged batteries to course staff.
- Report any burnt electrical fuses to course staff.

Lecture 1

ROS2

Robots are highly complex devices, requiring the coordination of various sensors and actuators in real time in order to achieve a specific goal. The advent and growth of modern software tools has created a multitude of opportunities for the field of autonomous robotics. Here, we'll be focusing on robotic *middleware*: a collection of software tools existing in between the computer's operating system (OS) and the actual robotics control software. The industry standard in robotic middleware is a program called *Robot Operating System*, or ROS. ROS is an open-source robotic middleware program used widely in the autonomous robotics field. A few years ago, the developers of ROS decided to rewrite the program, taking what worked and repairing what didn't... and they came up with ROS2.

1.1 Purpose of ROS2

Think about ROS2 as digital “glue”, connecting hardware components to a collection of concurrently running software processes called *ROS nodes*. Figure 1.1 is an abstract representation of ROS2 and demonstrates how it connects hardware components and software nodes.

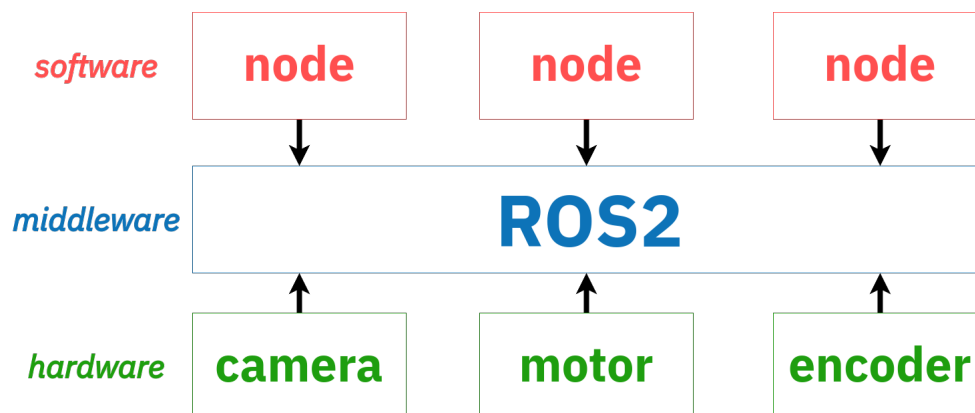


Figure 1.1: ROS2 as Middleware

Throughout this class, you'll hear people refer to ROS2 simply as ROS. Though ROS is

technically the first iteration of the software and ROS2 the second iteration, the two names are nearly always interchangeable.

1.2 The ROS Graph

All ROS nodes exist in an abstract structure called the *ROS graph*. The ROS graph is a collection of nodes running concurrently and communicating via named buses called *ROS topics*. Figure 1.2 shows an example ROS graph. The red circles represent ROS nodes, and each black arrow represents a topic. Nodes *publish* to topics by transmitting information to them, and nodes that *subscribe* to topics are notified when new information is available on the topic. In the figure, each arrow's tail corresponds to a publisher, and each arrow's head corresponds to a subscriber. Importantly, nodes may publish and subscribe to multiple different topics simultaneously. For example, a node responsible for drive motor control might subscribe to a topic called `drive_speed`, from which it receives information about how quickly to turn the left and right wheels.

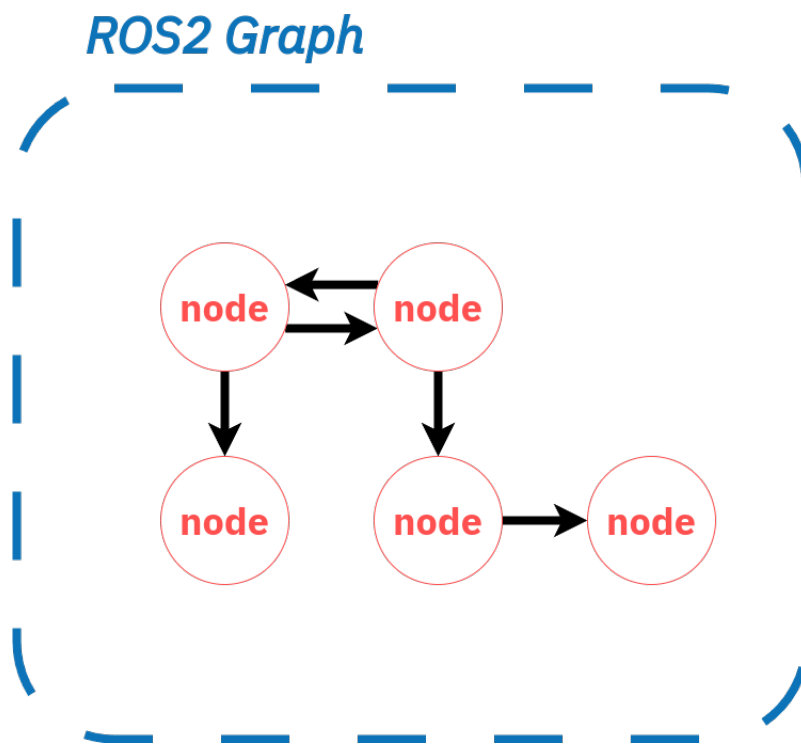


Figure 1.2: An Example ROS Graph

1.3 Messages

Information flows from node to node in the form of *ROS messages*. Messages take the form of a data structure composed of one or more *fields*. Figure 1.3 shows the format of a hypothetical `DriveCmd` message, which stores information about desired left and right wheel speeds. Each `DriveCmd` message has two fields: `l_speed`, the desired left wheel speed, and `r_speed`, the desired right wheel speed. Each field is represented by a 32-bit floating-point number (the data type `float32`). This only serves as an example; as the robotics engineer, you may select the name and structure of your messages as you see fit.

```
DriveCmd {  
    float32 l_speed,  
    float32 r_speed  
}
```

Figure 1.3: A ROS Message

1.4 Playing FizzBuzz with ROS

FizzBuzz is a simple math game that is widely used to demonstrate simple computer science concepts, including loops and conditional statements. The rules for FizzBuzz are very simple.

1. Beginning with the number 1, count upwards.
2. If the current number is divisible by three, say “Fizz”.
3. If the current number is divisible by five, say “Buzz”.
4. If the current number is divisible by three and by five (in other words, divisible by fifteen), say “FizzBuzz”.
5. If the current number is neither divisible by three nor by five, say the number.

Every game of FizzBuzz begins like so.

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

The game continues until some maximum number is reached. For our very first program, let's create a ROS graph that can play FizzBuzz! You may refer to the FizzBuzz tutorial on the MASLAB GitHub page, located at MASLAB/ROS2_FizzBuzz, to get started.

1.5 The ROS2 Ecosystem

The ROS2 ecosystem is a powerhouse of software packages, tools, and concepts that allows robotics engineers across the world to advance the state of the art in autonomous robotics. The FizzBuzz example above only serves as a very basic introduction to ROS2, and does not even begin to cover everything that the ROS2 ecosystem has to offer to help you along your adventure through this field. During the *Sensors & Visualization* lecture, we'll revisit ROS and introduce you to some more advanced tools that will accelerate your development processes and open the door to some very powerful ways of understanding the robotics design process.

Lecture 2

Version Control

Modern robots require *lots* of software. The software engineering process is extremely flexible and highly iterative, which means that it typically generates massive amounts of code across a multitude of versions in a variety of different languages. Most of these versions you'll never use; you'll only use the final working version after you've fixed all the bugs. But what if, as you're fixing a bug along the way, you break something that used to work? Or what if your new feature isn't nearly as helpful as you thought it'd be? Wouldn't it be nice if you could somehow keep track of every version in a centralized way?

2.1 Notation

This lecture will include a number of example Git commands. You will typically see the commands written as follows.

```
$ git subcommand ...
```

Note that the symbol \$ purely indicates that the following sequence is a terminal command... so only type what's written after it!

2.2 Version Control with Git

Software engineers have a variety of version control methods. In MASLAB, we use Git, which is far and away the most popular software version control system. Git is a piece of software that runs on your computer and keeps track of your code, but it's also a philosophy and a way of thinking about version control. Here, we'll learn about the philosophy behind Git so we can use it most effectively.

2.3 Repositories

If a project is managed by Git, the project is referred to as a Git *repository*. A repository is a directory (folder) on your computer containing files and other directories (folders). Funda-

mentally, a repository is a collection of commits and the set of relationships between them. To create a repository in a directory, open a terminal in that directory and type

```
$ git init
```

This will create a brand-new Git repository in the current directory.

2.4 Commits

The fundamental unit of version control in Git is the *commit*. A commit can be thought of as a “snapshot in time”: it captures the entire state of a repository at a given moment. This includes the contents of all files and the structure of the repository. Importantly, each commit does not store the full contents of the repository, but rather the *change* since the previous commit. This previous commit has a special name: the *parent commit*. For example, if you create commit **C1**, then create a file called `hello.txt`, and then create commit **C2**, commit **C2** would only contain the change “create file `hello.txt`”. Figure 2.1 provides a visual representation of a commit labeled **C1** (parent commit not shown).

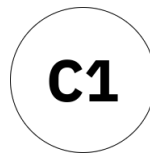


Figure 2.1: A Git Commit

To create a commit in Git, you must first *stage* all necessary changes. You can do this using `git add` and then typing the files or directories you’d like to stage. You can also stage everything for commit using `git add --all`. You can then commit changes like so.

```
$ git add --all
$ git commit -m "Commit message here!"
```

The sequence `-m` indicates that the following string is a *commit message*: a short English message describing the purpose of the commit. Remember to enclose your commit message in quotation marks!

2.4.1 Commit Hashes

Every commit has a unique identifier called a *commit hash*. When working with a Git repository, this is the only way to identify a commit. A commit hash looks something like

```
d6ef8b40d075c425f548002d2f35ae3f06e9cf96
```

This is a 40-character hexadecimal string, generated by a cryptographic hash algorithm based on the contents of the commit. Typically, to make life easier, you can just refer to this commit hash by its first 7 characters, like `d6ef8b4`. However, in this document, we'll use `C1`, `C2`, etc. to represent commit hashes.

2.4.2 Root Commits

Our commit `C1` seems a bit lonely, so let's give it some friends! As we build out our repository, we can't have history depend on itself, because that's impossible. In other words, every Git repository has to have at least one commit with zero parents, which is referred to as a *root commit* of that repository. Repositories can have multiple root commits, though this is typically uncommon. Figure 2.2 shows a visual representation of a repository with three commits. In the figure, the blue commit `C1` represents a root commit in the repository. Notice that arrows point backwards in history towards parent commits.

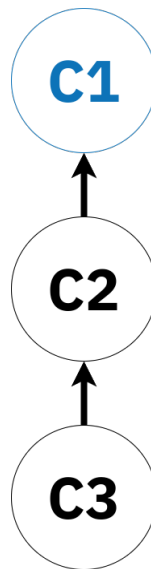


Figure 2.2: A Root Commit and Two More Commits

2.4.3 Merge Commits

Sometimes, two people have both done excellent work by creating two separate “histories” of work (two separate series of commits) and you'd like to merge their efforts into one unified result. This can be done by use of a *merge commit*. Merge commits are special in that they have two parent commits, instead of one. Git has an internal merging algorithm for constructing these commits, but sometimes it gets stuck and needs your help. We'll teach you how to merge commits later in this lecture. Figure 2.3 shows a visual representation of a repository with five commits. The red commit `C5` represents a merge commit, merging the changes produced in both `C3` and `C4`.

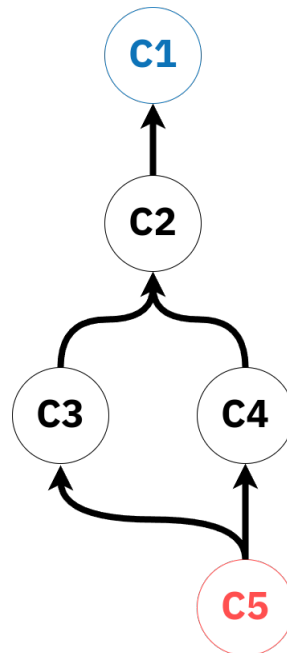


Figure 2.3: Creation of a Merge Commit

2.5 Branches

Commits are organized into higher-order structures called *branches*. We will define a branch as “a reference to a commit and all of its ancestors, up to and including the root commit.” Software engineers often name branches to make their lives easier. A branch’s name is typically a short lowercase alphanumeric string that describes its purpose. Every repository also typically has one very large branch (a “tree trunk”, perhaps) from which all the other branches... well, “branch”. Developers use a variety of names to refer to this branch, including `master`, `main`, and `trunk`. Figure 2.4 shows a repository with five commits and two branches, `main` and `bugfix`. The branch `main` contains the commits C5, C4, C2, and C1, whereas branch `bugfix` contains the commits C3, C2, and C1.

Let’s say we’re on commit C3 and we want to create the `bugfix` branch. We can do so easily by typing

```
$ git branch bugfix
```

This will create a new branch at C3. However, in order to do work on this branch, we first must *check out* the branch.

2.6 Checkout

Git repositories vary in size between very small software packages with maybe a dozen commits to extremely large projects with hundreds of thousands of commits. Each of these

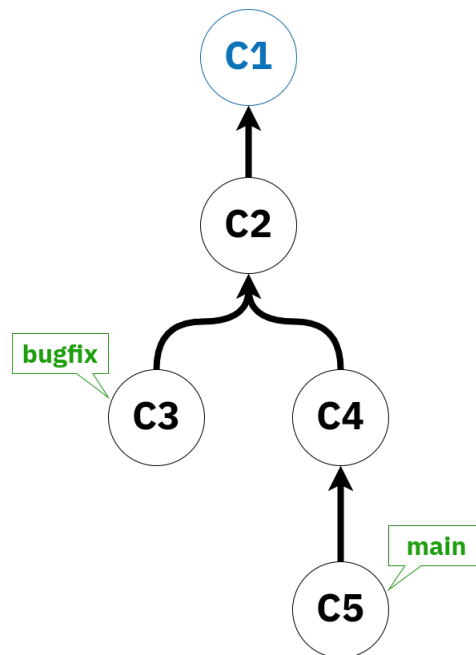


Figure 2.4: Two Branches

represent a distinct state of your repository. How do you know which one is reflected in your current project directory? Enter **HEAD**. **HEAD** is a reference to the commit that you're currently working on. In the business, software engineers say that **HEAD** is a reference to the commit that is currently *checked out*. When you ask Git to check out a specific commit, you change the files present in your project directory to reflect the state of that commit. This is a bit of a strange concept at first, so let's try an example! Figure 2.5 shows a repository with four commits checked out to the **main** branch. Notice that the reference **HEAD** points to **main**. To get here, we can type

```
$ git checkout main
```

As mentioned before, remember that after creating a new branch, you'll need to check out to it before working on it. For example, we could create a new branch called **feature** on **C4** and check out to it by typing

```
$ git branch feature
$ git checkout feature
```

But back to our original scenario. What happens when we commit? **HEAD** always moves to the new commit, and because we're checked out to a branch, we take **main** along with us! Take a look at Figure 2.6. Notice that we've created a new commit, **C5**, and the branch **main** has moved along with **HEAD** to the new commit.

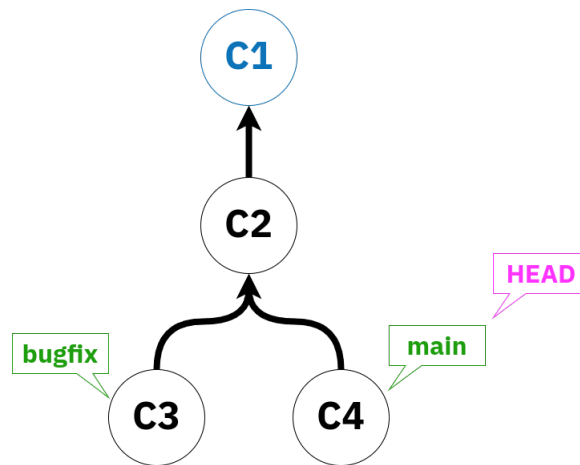


Figure 2.5: Checking Out to Main Branch

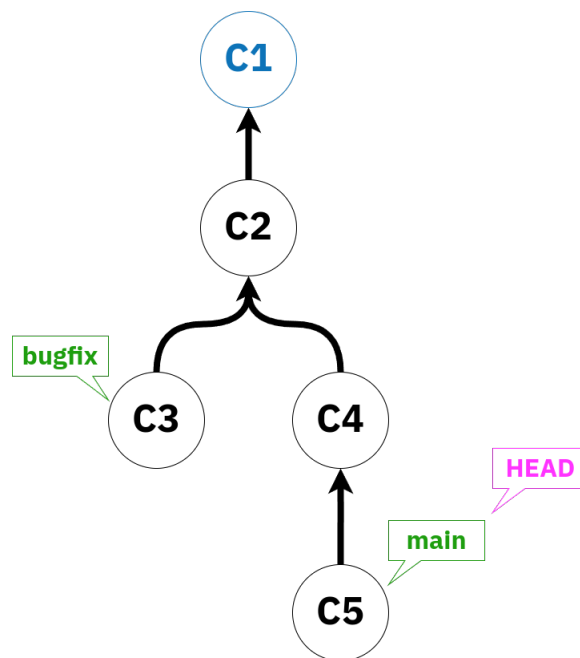


Figure 2.6: Committing to a Checked-Out Branch

When we're checked out to a branch, and that branch moves along with us when we create new commits, we consider **HEAD** to be *attached*. We won't always find ourself in this situation, though! What if we want to go back in history and read our old code? Git allows us to check out to any commit we want, so let's check out to commit **C1**. In Figure 2.7, you can see that **HEAD** has moved to **C1**. Because we're not checked out to any branch right now, we consider **HEAD** to be *detached*. To get here, we can type the following. Notice that Git politely notifies us that we've detached **HEAD**.

```
$ git checkout C1
```

You are in 'detached HEAD' state...

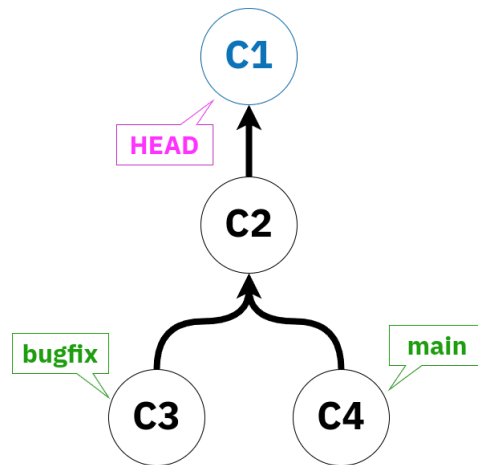


Figure 2.7: Checking Out to a Previous Commit

When you commit while **HEAD** is detached, you create a commit that's not on any branch. For example, let's check out *directly to commit* C4. This is different than checking out to **main**, as you can see in Figure 2.8. Notice that, instead of **HEAD** pointing to **main**, it points directly to C4. This is still considered a detached state, because we will no longer move **main** along with us when we commit. We can get into this detached state through terminal by typing

```
$ git checkout C4
```

You are in 'detached HEAD' state...

For demonstration purposes, let's try committing in this state. What happens? Take a look at Figure 2.9. As you can see, we've created commit C5, but because we're in a detached state, we didn't take branch **main** along with us! The terminal commands corresponding to this are

```
$ git checkout C4
$ git commit -m "Commit with detached HEAD"
```

Notice what this means... commit C5 is not a member of any branch! We refer to commits in this state as *orphan commits*. Typically, you don't want to have these in your repository, because they can be nearly impossible to locate and work with, especially in a large commit tree. Furthermore, branches are extremely easy to create, so the best thing to do in this situation is to create a new branch at C5.

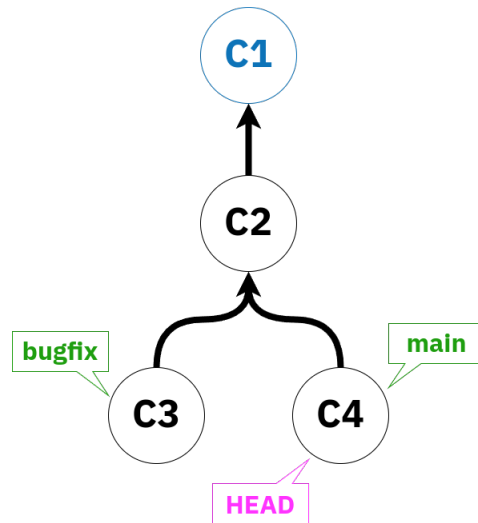


Figure 2.8: Detaching HEAD

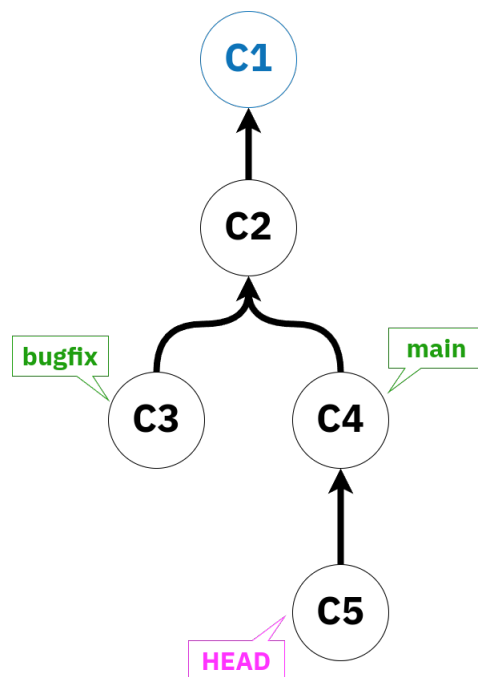


Figure 2.9: Creating an Orphan Commit

2.7 Merges

Let's say Alice and Bob are working on the repository in Figure 2.4 together. Bob is committing to `main` while Alice is fixing a bug introduced in `C2` by creating the branch `bugfix`. Alice has successfully fixed the bug and now wants to *merge* her work into the rest of the source code, in such a way that includes both Bob's work and Alice's fix. We can do this by

first checking out to `main` and then *merging* `bugfix` into `HEAD`. This process creates merge commit `C6`, as you can see in Figure 2.10. The state of the repository at `C6` includes both Bob's new work and Alice's fix. Notice that the branch `main` has now advanced to include commit `C6`. Bob can do this in his terminal by typing

```
$ git checkout main
$ git merge bugfix
```

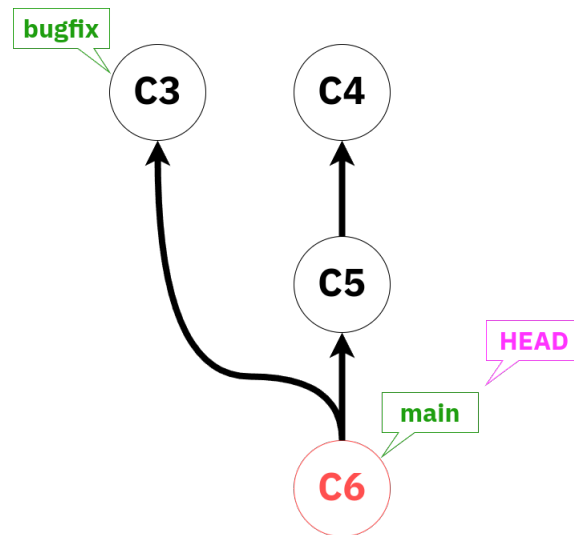


Figure 2.10: Merging Alice's Fix into the Main Branch

2.8 Merge Conflicts

It's important to note that merging can sometimes be a complicated process. Git has a built-in merging algorithm that will attempt to merge work across the various files in your repository, but it won't always succeed. If Git isn't successful in merging two commits automatically, you've entered a *merge conflict*. When you encounter a merge conflict, you have to open the conflicting files (Git will tell you which ones) and resolve the conflict manually. Merge conflicts occur in one of two situations.

1. Each of the two commits to be merged modify the same line of the same file.
2. One of the two commits to be merged deletes a file, while the other commit edits the file.

A file with one or more conflicting lines of code will look something like Figure 2.11. To resolve the conflict, you'll have to remove the lines shown here in red (their color will depend on your editor) and change the code contained within them to the desired merge

result. Then, you can commit the result manually. Git knows when it's in a conflicted state and will automatically create a merge commit from your manual changes. Resolving a merge conflict looks like the code below.

```
$ git checkout main
$ git merge bugfix

CONFLICT: ...

[fix conflict]

$ git add conflicted-file.txt
$ git commit -m "Merge bugfix into main"
```

```
<<<<<<< HEAD
existing content
=====
incoming change
>>>>>>> bugfix
```

Figure 2.11: An Example Merge Conflict

2.9 Fast-Forwards

Merge commits are created when there are two *divergent histories* that need to be reconciled into one commit. But what if two histories that never diverged need to be merged? This is a hard thing to think about, so let's use another example. Consider Figure 2.12, in which Alice has fixed a bug in `bugfix` but Bob hasn't done any additional work on `main`. We're currently checked out to `main` (`HEAD` points at `main`) and we want to merge the work on `bugfix` into the current branch. There's really no need for a merge commit, because all we want is to move `main` up to C4, right?

In this situation, we want to perform a *fast-forward merge*, in which Git will simply move the branch tip of `main` to include Alice's hard work in fixing the bug on `bugfix`. We call this a fast-forward (often abbreviated as FF) because `main` is "fast-forwarding through

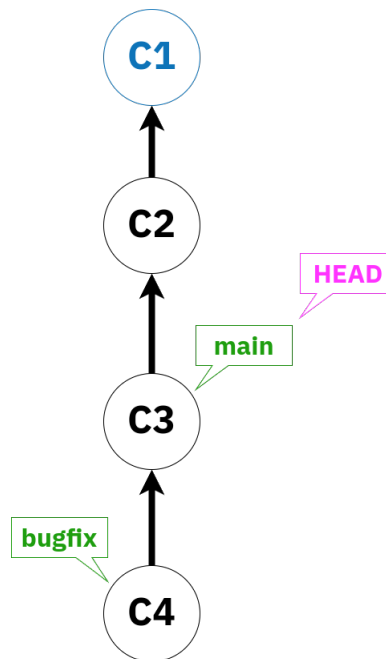


Figure 2.12: Before Fast-Forward

history” without requiring any actual merging. To perform a fast-forward merge, we just need to check out to `main` (already done) and merge `bugfix`. Git will automatically detect that a fast-forward is required and move the `main` branch tip for us. Figure 2.13 shows the repository’s state after the fast-forward merge is completed. Notice that, unlike a regular merge operation, we don’t create a merge commit. This process is no different in the terminal than a regular merge, except that Git politely informs us that we are doing a fast-forward merge instead of a regular merge.

```
$ git checkout main
$ git merge bugfix
```

```
Fast-forward ...
```

2.10 Advanced Features

Git is an extremely powerful software tool in the hands of an experienced developer. This introduction only scratches the surface of Git’s features, and doesn’t cover a number of more advanced features. To explore these features and get some practice with Git, we encourage you to check out the excellent interactive tutorial located at <https://learngitbranching.js.org/>.

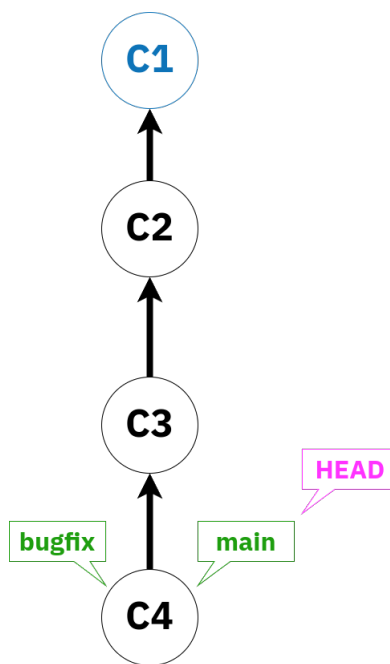


Figure 2.13: After Fast-Forwarding

Lecture 3

Kitbot

Building a fully autonomous robot is *hard*. Doing it in a month is nearly impossible. Nothing you can't handle... because nothing's really impossible, right? To assist you along your journey, the course staff has been preparing for you a lab kit, from which you'll construct Kitbot. Kitbot is a very rudimentary robot that can take pictures and drive around on the game field. This lecture is focused on orienting you to your Kitbot and its constituent hardware and software. Over the next three weeks of MASLAB, you'll be preparing your Kitbot for the great Final Competition during the last week of IAP.

3.1 Architecture

Your Kitbot is composed of components across the three domains of the field of robotics: the mechanical, electrical, and software domains.

3.1.1 Mechanical

Chassis Your robot's chassis is cut from one-half inch medium-density fiberboard (MDF). MDF is created under high pressure from wood particles and a resin-based binder. It's an inexpensive but relatively sturdy material that's easy to cut on a laser cutter and perfect for robots made with rapid prototyping technologies, such as Kitbot.

Motors Two 12-volt DC geared motors keep your robot rolling along. These consume most of the power on your robot, and are precisely controlled by the Kitbot electrical hardware. Your robot will use these motors to navigate across the game field, and we'll learn more about path planning and robot kinematics in Lecture 7.

Vision A Logitech Brio 101 webcam gives your Kitbot eyes to see the world. This high-definition camera has 2.1 million color pixels... 2.1 million opportunities for Kitbot to see the game field before it. Lecture 8 will cover the fundamentals of computer vision technology, which will allow your Kitbot to make sense of your camera's pictures.

3.1.2 Electrical

Processor A Raspberry Pi 5 makes up the brain of your Kitbot. With a 2.4 GHz processor, 8 GB of RAM, and 64 GB of solid-state storage, and sporting an air-cooled central processor, this little credit card-sized computer will handle every autonomy task you give it.

Controller The Raspberry Pi 5 is powerful, but it can't handle the high currents required to turn your motors. Enter the Raven controller board. The Raven was designed in-house by MASLAB alumnus and TA Miguel Flores-Acton, and it's responsible for providing DC power and precise control to the drive motors at sufficient current.

Battery To drive on its own, your robot needs its own power source. This is the job of its 5.2 ampere-hour lithium-polymer battery, powering Kitbot towards victory like the Energizer bunny. We'll talk more about the battery a little later in this lecture.

3.1.3 Software

Operating System Your Raspberry Pi 5 runs Ubuntu Linux. This is the official operating system for Kitbot's middleware, ROS2.

Middleware Robot Operating System 2 (ROS2) is the industry standard in robotic middleware. ROS2 brings with it a whole toolbox of very powerful software packages that we'll use throughout the course. Lecture 1 covered the fundamentals of ROS, but you'll learn more about it as the course continues.

Image Processing Python's OpenCV library will allow Kitbot to make sense of the images coming in from its HD webcam. Lecture 8 will cover the fundamentals of computer vision, and Lecture 9 will show how recent technological advancements in machine learning can strengthen your robot's vision capabilities.

Lecture 4

Sensors & Odometry

Your robot will use multiple sensor devices to estimate its position in space. How can we combine the information from these multiple sensors in a meaningful manner in order to understand its position?

4.1 Kinematics

To estimate the position of a robot in space given a number of inputs, we will use a *statistical inference* technique. Inference allows us to rigorously understand the relationship between *signal*, which is meaningful information, and *noise*, which is meaningless information that becomes incorporated into the signal. Let's introduce a simple example to think through this process of inference. Suppose we know that at some time $t = t_n$, the robot is at position X_n . The robot also has some velocity V_n . We can see a recurrence relationship between X_n and X_{n+1} in (4.1).

$$X_{n+1} = X_n + V_n \Delta t \quad (4.1)$$

This is a very simple kinematic relationship. Now, it would be great if we knew the velocity and position of the robot exactly, right? If so, we could just measure it exactly and be done with this. However, in reality, we almost never know these values exactly. Let's suppose that we can measure the velocity as Y_n , which contains some velocity measurement error W_V .

$$Y_n = V_n + W_V \quad (4.2)$$

Here, V_n is the signal and W_V is the noise, but we can only know their combination Y_n through observation. Let's also suppose that we can measure the position of the robot as Z_n in a similar way, with a position measurement error W_X .

$$Z_n = X_n + W_X \quad (4.3)$$

How can we estimate X_{n+1} from these two potentially erroneous measurements Y and Z ?

4.2 Estimators

We will use $\hat{X}_{n+1}$ to denote our “best guess” of X_{n+1} ... statisticians call this an *estimator* of X_{n+1} . Below is one proposed estimator. We’re estimating our change in position to be a weighted sum between our measured change in position $Z_n - \hat{X}_n$ and our calculated change in position based on our measured velocity $Y_n \Delta t$.

$$\hat{X}_{n+1} = \hat{X}_n + \alpha(Z_n - \hat{X}_n) + (1 - \alpha)Y_n \Delta t \quad (4.4)$$

The weighting factor in the sum is α , which tells us something about how much we “trust” each of our measurements. If we know that the position measurement is more accurate than the velocity measurement, α would be close to one, but if the velocity measurement is more accurate, then α would be close to zero. We can select this value carefully to maximize the effectiveness of our estimator.

4.3 Estimation Error

How can we select α to maximize our estimator’s effectiveness? Well, to maximize an estimator’s effectiveness, we can *minimize the magnitude of its error*. The error of an estimator is defined to be as follows.

$$\varepsilon_{n+1} = X_{n+1} - \hat{X}_{n+1} \quad (4.5)$$

Obviously, we’d like this quantity to be as close to zero as possible. If ε is very positive or very negative, then our estimator is missing the mark. In order to quantify how far our error is deviating from zero (the ideal), let’s square it.

$$(\varepsilon_{n+1})^2 = (X_{n+1} - \hat{X}_{n+1})^2 \quad (4.6)$$

Let’s substitute known quantities and see where this takes us!

$$(\varepsilon_{n+1})^2 = (X_n + V_n \Delta t - \hat{X}_n - \alpha(X_n + V_n \Delta t + W_X - \hat{X}_n) - (1 - \alpha)(V_n + W_V) \Delta t)^2 \quad (4.7)$$

Woah, that’s a lot of terms. Fortunately, we can clean this up significantly by grouping or cancelling many of the terms like so.

$$(\varepsilon_{n+1})^2 = ((X_n - \hat{X}_n)(1 - \alpha) - \alpha W_X - (1 - \alpha)W_V \Delta t)^2 \quad (4.8)$$

Take a look at that first term. Isn’t that the error from the previous time step?

$$(\varepsilon_{n+1})^2 = (\varepsilon_n(1 - \alpha) - \alpha W_X - (1 - \alpha)W_V \Delta t)^2 \quad (4.9)$$

That’s definitely more tame. Remember that we want to minimize the error... how can we do that? We don’t know the values of W_X and W_V because they’re measurement noise and therefore inherently unknowable. However, we assume that we do know the measurement’s expected value (zero) and its variance (nonzero). Therefore, let’s minimize the *expected value*

of the error. This quantity, called the *mean squared error* or MSE, is critical to determining optimal estimators. We'll use the symbol $\mathbb{E}[\cdot]$ for expected value here.

$$\text{MSE}_{n+1} = \mathbb{E}[(\varepsilon_n(1 - \alpha) - \alpha W_X - (1 - \alpha)W_V \Delta t)^2] \quad (4.10)$$

Expected value is linear, so let's evaluate the square and then distribute the expected value over each resultant term.

$$\begin{aligned} \text{MSE}_{n+1} = \mathbb{E}[\varepsilon_n^2(1 - \alpha)^2 + \alpha^2 W_X^2 + (1 - \alpha)^2 W_V^2 \Delta t^2 \\ - \varepsilon_n \alpha(1 - \alpha)W_X - \varepsilon_n(1 - \alpha)^2 W_V \Delta t + \alpha(1 - \alpha)W_X W_V \Delta t] \end{aligned} \quad (4.11)$$

As we distribute the expected value operator, we remember that our measurement errors have zero expected value, even though they are rarely zero. This is because we assume that the error is distributed symmetrically around zero. This means any term with W_X or W_V (but not their squares!) becomes zero.

$$\text{MSE}_{n+1} = \mathbb{E}[\varepsilon_n^2](1 - \alpha)^2 + \alpha^2 \mathbb{E}[W_X^2] + (1 - \alpha)^2 \mathbb{E}[W_V^2] \Delta t^2 \quad (4.12)$$

We will also use the fact that, for distributions with zero expected value, $\text{var}(X) = \mathbb{E}[X^2]$.

$$\text{MSE}_{n+1} = \mathbb{E}[\varepsilon_n^2](1 - \alpha)^2 + \alpha^2 \text{var}(W_X) + (1 - \alpha)^2 \Delta t^2 \text{var}(W_V) \quad (4.13)$$

Look at that first term again. We recognize that as the MSE from the previous time step!

$$\text{MSE}_{n+1} = \text{MSE}_n \cdot (1 - \alpha)^2 + \alpha^2 \text{var}(W_X) + (1 - \alpha)^2 \Delta t^2 \text{var}(W_V) \quad (4.14)$$

We've now successfully determined the mean squared error of our estimator for each time step in terms of quantities that we know as well as α , which we can choose. How shall we select it?

4.4 Optimizing Estimators

We want to select the estimator that has the smallest possible mean squared error. In the world of statistical inference, statisticians call this the *least mean square* estimator, or LMS estimator. We'll compute the LMS estimator by determining the value of α that makes the MSE as small as possible.

$$\frac{\partial}{\partial \alpha} \text{MSE}_{n+1} = 2\text{MSE}_n \cdot (\alpha - 1) + 2\alpha \text{var}(W_X) + 2(\alpha - 1) \Delta t^2 \text{var}(W_V) = 0 \quad (4.15)$$

Solving for α , we can calculate its optimal value α^* .

$$\boxed{\alpha^* = \frac{\text{var}(W_V) \Delta t^2 + \text{MSE}_n}{\text{var}(W_X) + \text{var}(W_V) \Delta t^2 + \text{MSE}_n}} \quad (4.16)$$

Finally, we have an expression for an optimal value of α that will minimize our MSE! We can experimentally determine the variances of our measurement methods by measuring a value we know is constant, and we can keep a running estimate of our MSE using (4.14). To demonstrate how powerful this estimator is, let's think about a few example cases.

- Suppose that we know our velocity measurement is very noisy. This will cause $\text{var}(W_V)$ to be very high, driving up α and forcing us to rely more on our position. We do this to prevent drift. If we relied on our velocity too heavily while we are receiving noisy measurements, our estimator could drift significantly over time and we'd have no way of knowing.
- Suppose now that we're in the opposite scenario, and our position measurement is more noisy. This will cause $\text{var}(W_X)$ to be very high, driving down α and forcing us to rely more on our velocity measurement. However, the running estimate of MSE is going to grow a lot as well, because the $(1 - \alpha)^2$ factor on the previous step's MSE will be close to one. As MSE grows, it's going to drive α back up just a bit. It does this again because small errors in the velocity measurement may cause significant drift over time, and the estimator begins to rely just a bit more on position to correct for this.
- Suppose we're taking measurements across large time steps. Small errors in velocity measurement are amplified by this, but small errors in position measurement are not amplified. Therefore, we want to rely more on our position measurement. We see this reflected in the relation for α because it grows as Δt grows.

4.5 Visualizing Estimators

To demonstrate the power of our LMS estimator, let's look at a demonstration. Suppose our robot will travel with the following velocity profile. The velocity is in centimeters per second.

$$v(t) = \begin{cases} 10t & 0 \leq t < 5 \\ 100 - 10t & 5 \leq t < 10 \\ 0 & \text{otherwise} \end{cases} \quad (4.17)$$

Suppose we also know that $\text{var}(W_X) = 625 \text{ cm}^2$ and $\text{var}(W_V) = 100 \text{ cm}^2/\text{s}^2$. In other words, on average, the position measurement deviates by 25 centimeters and the velocity measurement deviates by 10 centimeters per second. Finally, we'll set our time step to be $\Delta t = 10 \text{ ms}$. Take a look at the velocity and position measurements, as well as the LMS estimator in orange, in Figure 4.1.

To demonstrate the power of the estimator, we can calculate the square root of the MSE, which will give us the average distance by which the estimator deviates. This is called the *root mean square error*, or RMSE. You can see that here, even though the position measurement deviated by up to 25 centimeters and the velocity measurement by 10 centimeters per second, our LMS estimator keeps the RMSE down to less than 2 centimeters over a distance of over 2 meters... less than 1% error!

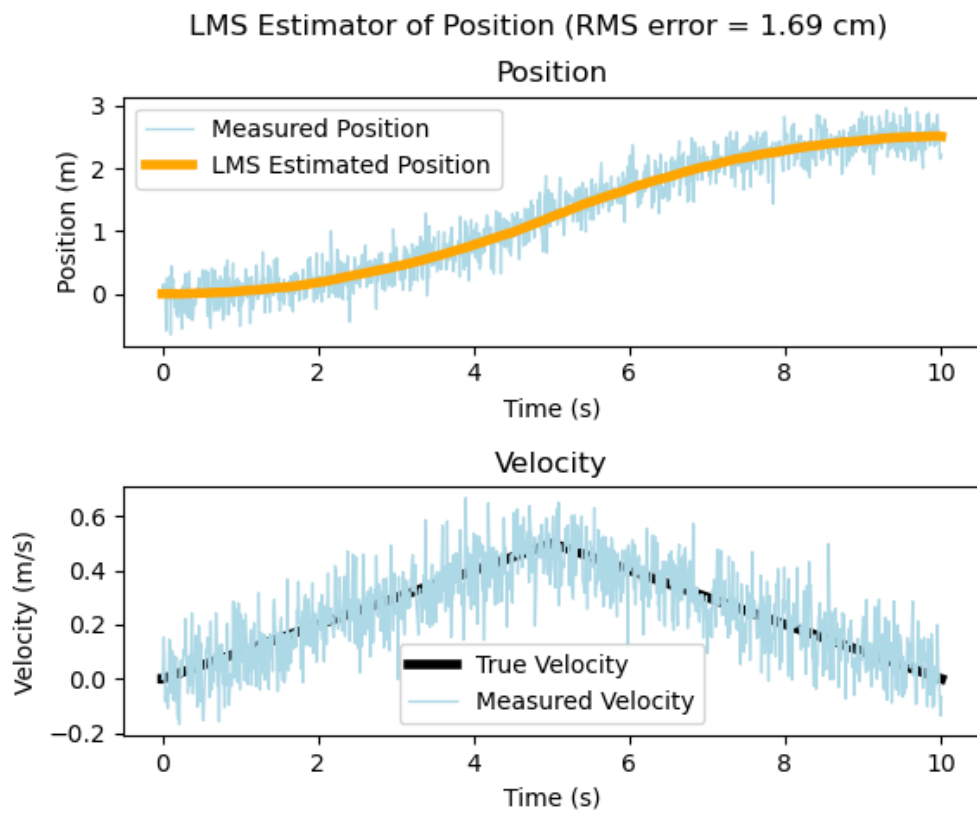


Figure 4.1: Estimating Position from Noisy Measurements

Lecture 5

Principles of Design

Mechanical, electrical, and software engineering are iterative processes, requiring creativity, flexibility, experience, skill, and open-mindedness. As your projects grow more complex, it becomes increasingly important to keep your work organized and reliable. This lecture provides a distillation of decades of practical wisdom from the industry. We encourage you to implement these practices according to your best judgment in your development processes, to best ensure the robustness and reliability of your engineering work.

5.1 Mechanical

5.1.1 Mechanical Constraints

When developing mechanical systems, it's critical to think about *mechanical constraints* and how they interact with various components. Let's start with a classic example, to get us thinking about the idea of constraining things in space. Suppose you're given the task of designing a stool. How many legs should you put on the stool? One is surely not enough, or anybody who tries to sit on it would fall right away. Two is also not enough, for the same reason. When a mechanical system is free to move more than we would like, we say that the system is *underconstrained*.

So more constraints is better, right? Not quite... have you ever experienced sitting on a four-legged stool or chair, but one of the legs isn't the same length as the others, and you rock back and forth? It can be a frustrating experience. When a mechanical system has more constraints than it should, we say that the system is *overconstrained*. I've hinted at it, but the optimal number of legs for a stool, from a perspective of mechanical constraint, is three. This is because stools with three legs simply cannot rock back and forth like stools with four or more legs. In accordance with this principle, when designing your robot, ensure that every part has sufficient constraints that it won't move around more than you want, but not too many constraints, or you might not be able to attach it to the rest of your robot at all!

5.1.2 Physical Compliance

Robotic systems, and other various mechanical systems, benefit greatly from *compliant parts*. The use of compliant mechanisms is an excellent way to avoid overconstraining robotic systems. Compliant mechanisms are physically soft and deform slightly due to their flexibility. One of the many benefits afforded by compliant mechanisms is the lack of need for exact precision in actuation, which is always subject to error and measurement noise. For example, to pick up a wooden game piece, you could use a foam gripper rather than one made from solid plastic. A solid plastic gripper will have less contact area, and if you use too much force to grip the cube, you could crack the gripper! These concerns are greatly mitigated with a foam gripping surface, which will deform to produce a large contact area and can accommodate a range of forces without cracking.

5.2 Electrical

Voltage and current are used for a nearly unfathomable multitude of purposes in robotic systems. Electrical systems can be used to transfer energy, communicate information, perform computation, and coordinate the behavior of various sensors and actuators. A very simple and highly effective way to organize these various applications is *wiring convention*. Wiring conventions can be very flexible, and there's no single convention that everybody follows. Take a look at Figure 5.1 for an example convention. Note that each type of wire is associated with both a *symbol* (e.g. 3V3 for 3.3 volts) and a *color* (e.g. orange for 3.3 volts). We encourage you to agree with your team upon a convention that you find intuitive, and remain consistent with that convention throughout the course. This will prevent you and your teammates from creating short circuits or shocking your electronics with excessively high voltage accidentally.

<i>red</i>	<i>5V</i>	<i>5 volts</i>
<i>blue</i>	<i>12V</i>	<i>12 volts</i>
<i>orange</i>	<i>3V3</i>	<i>3.3 volts</i>
<i>black</i>	<i>GND</i>	<i>ground</i>

Figure 5.1: An Example Wiring Convention

5.3 Software

5.3.1 Finite-State Machines

How do robotic systems make decisions autonomously? Typically, robots collect information from their environment and combine this information with their current behavior to determine the best next action. This behavior is perfectly encoded in the concept of a *finite-state machine* or *FSM*. In a finite-state machine, your robot can exist in one of a finite number of *states*, and a variety of internal and external stimuli can prompt the robot to transition into another state. See Figure 5.2 for an example finite-state machine for a very simple robot. This FSM has three states, and the robot transitions between these three states depending on its current state and interactions with its environment. When you implement the FSM in your robot's software, you can use a while loop and a series of conditional statements to direct the behavior of the robot based on each state. Below is a simple Python-like pseudocode implementation of the FSM diagram in Figure 5.2.

```
# Start in the "search" state
state = "search"

while time_left > 0:
    # Search for the block
    if state == "search":
        if see_block():
            state = "drive"

    # Drive to the block
    elif state == "drive":
        drive_to_block()
        state = "pickup"

    # Pick up the block
    elif state == "pickup":
        pick_up_block()
        state = "search"

# We've left the loop, so time's up
shutdown_robot()
```

5.3.2 Version Control

Robots require *lots* of software. Complex robotic systems require hundreds of thousands, if not millions, of lines of code. When multiple developers, or multiple teams of developers, are concurrently working on these software projects, it becomes absolutely vital to organize the flow of development into a structured and meaningful process. Software *version control*

practices enable developers to keep this structure in their workflow. We covered the fundamentals of version control using Git in Lecture 2, and we strongly encourage you to use the Git philosophy and software program to organize and structure your software development process. You may always refer back to the slides and notes from that lecture to strengthen your understanding of the Git version control system.

5.3.3 Naming Conventions

Software development involves the creation and use of countless variables, functions, objects, methods, as well as the generation and handling of log messages and errors. Thoughtful *naming conventions* can make the difference between an indecipherable “alphabet soup” and a software program that’s easy to understand, improve, and deploy. Names are effective when:

1. You immediately understand the purpose of something by only reading its name, and
2. You can easily read, type, and pronounce the name.

The effectiveness of a variable name is also shaped by the case of its application. For example, in a computer program, the variable name `x` is an excellent name for a number representing a position, because the coordinates (x, y, z) are commonly used to refer to points in Cartesian space. However, `x` is a very poorly selected name for a string representing the current state of the robot in a finite-state machine, because the name is not at all descriptive. Being too descriptive is just as bad though... imagine trying to type out `current_state_that_the_robot_is_in` every single time! Thoughtfully selected variable names, such as `state` or `current_state` are far easier to read, type, and pronounce. This principle applies to error messages as well. If your software returns error messages like

Error

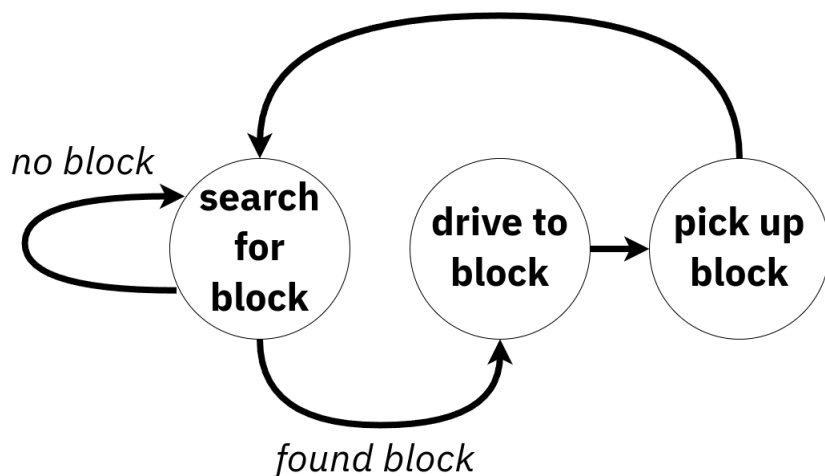


Figure 5.2: A Simple Finite-State Machine

then how do you know what went wrong? You might be debugging for hours! Writing log messages and error messages that are easy to understand will transform the engineering process from agonizingly slow and headache-inducing to streamlined, smooth, and pleasant. We invite you to consider the following examples when writing error messages... these are far more helpful and will reduce your debugging time from hours to minutes.

```
Error: file 'exapmle.txt' not found, is it spelled right?
```

```
Error: variable 'state' has invalid value '35.0'  
      (remember, 'state' should be a string)
```

```
Error: could not connect to camera on port '/dev/ttyUSB0',  
      maybe try another port?
```

5.3.4 Sematic Versioning

Version control practices are highly effective for structuring the software development process. However, software never exists in isolation. Software packages always need to interface with operating systems, middleware programs, and hardware components. On top of version control processes, robust *interface management* practices are indispensable to ensuring that your robot continues to function effectively throughout the rapid and highly flexible process of software development. The industry standard in interface management is the *Semantic Versioning* standard, also known as *SemVer*. The SemVer standard was originally developed by Tom Preston-Werner, the cofounder of GitHub. The full standard is accessible on its official website at <https://semver.org/>. Here, we'll just discuss its basic principles, but we encourage you to read the standard itself if you'd like to learn more! Version numbers created using the SemVer standard are composed of three integers, separated by decimal points. For example, 1.3.12 is a valid SemVer version. The first number is referred to as the *major version*, the second number the *minor version*, and the third number the *patch version*. You can see this visualized in Figure 5.3.

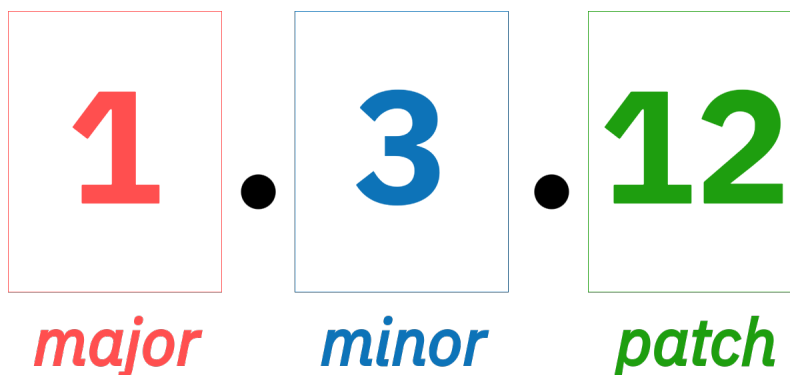


Figure 5.3: Fundamentals of the SemVer Standard

Major Version The major version is incremented every time a *breaking change* is made to a software interface. Breaking changes violate the principle of *backwards compatibility*. Backwards compatibility requires that new versions of software packages be able to seamlessly replace their older counterparts, without requiring updates to other connected software packages. For example, if a software package includes a function `foo`, it would be considered a breaking change to remove function `foo`, because other software outside that package may depend on that function! Whenever the major version is incremented, *the minor and patch versions are reset to zero*.

Minor Version The minor version is incremented every time a *backwards compatible change* is made to a software interface. For example, if a software package includes a function `foo`, it would be considered a backwards compatible (non-breaking) change to add function `bar`, because no dependent software would break with the addition of a new function. Whenever the minor version is incremented, *the patch version is reset to zero*.

Patch Version The patch version is incremented every time a change is made that *does not change the software interface*. For example, if a software package includes a function `foo`, it would be considered a patch to refactor the source code of `foo` to make it more efficient, without changing its inputs or outputs. This is because no new interfaces were introduced, nor were any removed or changed.

5.4 Testbenches

After you've designed a robotic system, you want to make sure that it actually does what you designed it to do. Engineers develop *testbenches* to ensure that each component performs according to its original design, and that all elements work together to perform correctly. Fundamentally, a testbench is an environment designed for the purposes of verifying the design of a complex engineering system. Testbenches can be physical, such as an actual lab bench dedicated to testing, or abstract, consisting of a collection of processes, standards, and software packages. Critically, testbenches must contain both *unit tests* and *integration tests*.

5.4.1 Unit Tests

To ensure that each individual component or subsystem performs correctly, engineers design *unit tests*. Unit tests assess the design and performance of each individual “unit” of your robot. For example, a unit test developed to assess motor performance might instruct the robot to rotate in a full circle in the counterclockwise direction, and then back in the clockwise direction. While the test is running, you and your team can check visually that the robot in fact rotates 360 degrees in each direction, and not more or less, and that it doesn't drive forward or backward at all. We encourage you to develop an assortment of unit tests for driving, game piece manipulation, computer vision, and other subsystems, so you can maximize your confidence that each element of your design works as intended.

5.4.2 Integration Tests

Just because each of your robot's individual components perform correctly on their own doesn't mean your robot will work perfectly! For a complex robotic system, it's not enough to verify the performance of each component... you need to also verify the interfaces and integration across these components! For this purpose, engineers develop and run *integration tests*. Integration tests verify the performance of entire systems, and can only be used meaningfully after unit testing has verified the performance of each component. An example integration test might involve your robot driving forward towards a block, picking up the block, and stacking it on top of another block. This test verifies the performance and communication of your motor control, game piece manipulation, computer vision, and software control systems. In addition to your collection of unit tests, we encourage you to develop a library of integration tests, so that you may be confident that your robot will perform as intended on the competition field.

Lecture 6

Control Theory

Experience tells us that the real world is chaotic, messy, and unpredictable. Our sensors aren't always exactly correct, and neither are our actuators. If our robots are to fare well through the challenges of the real world, then we must design robust controllers that can withstand these difficulties.

6.1 Terminology

Before we get started, we'll define a bit of terminology.

Plant It's said that the foundations of control theory were first laid by chemical engineers, and their naming conventions have stuck. In control theory, a *plant* is a system we'd like to control. The plant receives input from a controller and returns an output to the environment. For example, a plant might be a drive motor; the motor receives a control signal from the motor controller, and it returns as output its spin speed and torque to the environment. In the diagrams in this document, you'll see the symbol G represent a plant.

Controller A *controller* is a system that we build that receives a setpoint and passes a *control effort* to the plant. In the diagrams here, you'll see the symbol K represent a controller.

Setpoint A *setpoint* is the output value you desire for the plant. The most common example of a setpoint is the temperature to which you set your thermostat. The setpoint, along with the controller, determines the control effort passed to the plant.

Error The *error* of a controllable system is defined as the difference between the output of the system and the setpoint of the controller. Error is *positive* when the output is too high and *negative* when the output is too low. In the thermostat example, error is positive when it's too hot, negative when it's too cold, and zero when it's just right.

6.2 Bang-Bang Control

The simplest and easiest form of control to implement is referred to as *bang-bang control*. Imagine it's the dead of winter in Massachusetts, and your dormitory doesn't have any form of heating. All you have is a cheap space heater (don't tell EHS!) with two settings: ON and OFF. How would you keep your room comfortable? One viable strategy is to turn the heater on until it gets uncomfortably hot, then turn the heater off until it gets uncomfortably cold, then repeat the process until the end of the semester. That's an awful lot of work, so let's say you go down to the makerspace and build a thermostat that automatically turns on and off your space heater when the temperature goes above or below a certain setpoint. This approach will work, in theory, with one major issue: this requires a significant amount of switching. Every time you switch it on, you waste valuable power through the space heater's internal capacitance and thermal resistance. Also, wouldn't it be nice if your room didn't cycle between uncomfortably hot or uncomfortably cold, but rather stayed comfortably warm? Surely there's a better way.

6.3 Open-Loop Control

You take the MIT shuttle to The Home Depot in Assembly Square and you get yourself a much nicer space heater that has a dial. If you set the dial to zero, the space heater turns off and produces no heat, and if you set it to any positive number, it will produce some amount of heat proportional to that number. Your input to the system (the number on the dial) then affects the temperature in your room in some way. This is reflected in Figure 6.1, in which K represents your *open-loop controller*, and G represents the *plant*. The plant contains your space heater's heating coil, the amount of air in your room, and the insulation in your walls, because all of these have properties that can affect the temperature inside your room. This is great, because now, you can just turn up your heater until it's warm and leave it all semester... right?

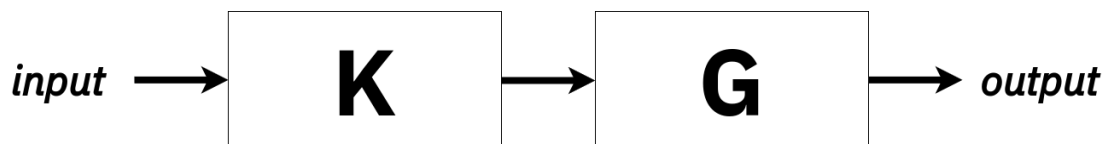


Figure 6.1: Open-Loop Feedback Control

Maybe if you're very lucky! In the real world, our understanding of complex systems is very limited, and we'd like our controllers to work despite this limitation. If the temperature outside changes at all, or if your roommate opens the window, or if the heater actually doesn't have a constant heat output, then the temperature in your room will fluctuate wildly! This is a pretty unreliable system, if you ask me. Critically, your heater has no way of accounting for the actual temperature in the room. Maybe we should actually account for the temperature in your room when we set the dial...

6.4 Closed-Loop Control

Open-loop control is better than bang-bang control, but it's still pretty weak, because it's incapable of accounting for any error that arises. In simple terms, open-loop controllers don't "know" when they're not doing it right. What if we made the controller's error itself an input to the controller? Let's go back to your space heater. You go back down to your dormitory makerspace and you rig up a little device that adjusts your space heater based on the current room temperature. This is reflected in Figure 6.2, in which the output is subtracted from the input to determine an *error*, which is then fed into the controller K .

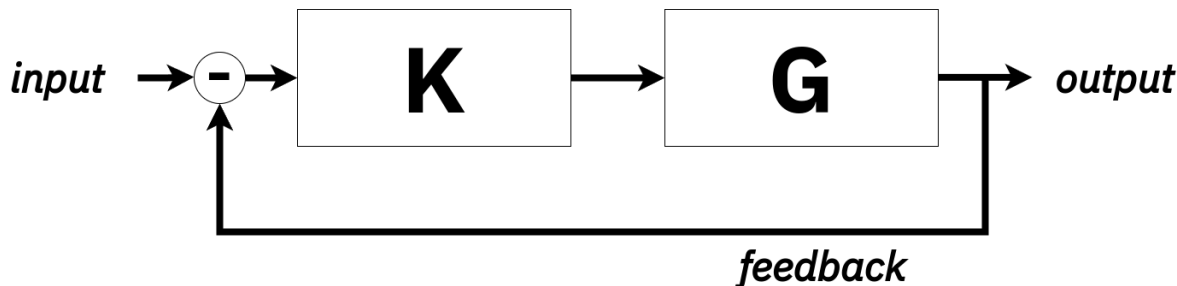


Figure 6.2: Closed-Loop Feedback Control

This is the critical step in *closed-loop feedback control*, in which the controller provides a control effort based on the plant output itself. It's important to note that bang-bang control is technically a form of closed-loop feedback control, but it's considered a very rudimentary method of control because of its two discrete possible outputs: ON and OFF.

6.5 Proportional Feedback Control

Let's think about designing a simple feedback controller for our dorm room's space heater. At an extremely basic level, how should the controller keep the temperature at the room close to the setpoint? Well, if the error (difference between the actual temperature and the setpoint) is positive and it's too hot, we want to turn off the space heater or even open a window, and if the error is negative, we want to turn on the space heater. Fundamentally, we want a response that is *proportional* to the error. We can write this as (6.1), where $k(t)$ represents the control effort (heat output by the space heater) and $e(t)$ represents the error. We can include the constant K_p as a constant of proportionality, which gives us a "dial to turn", allowing us to adjust the response of our *proportional feedback controller*.

$$k(t) = K_p e(t) \quad (6.1)$$

6.6 PD Feedback Control

Our proportional feedback controller is pretty effective. However, there's one problem with it... what happens when your roommate opens a window? The error will suddenly increase rapidly, and it will get pretty cold in the room. Our proportional feedback controller might eventually correct for this, but it could take a while, and the temperature might oscillate as the space heater constantly tries to correct and overshoots. We can fix this by introducing a term that resists *change in the system error*. Mathematically, we understand instantaneous change in the language of derivatives, so let's add a new term with constant K_d . Equation (6.2) describes our new *proportional-derivative feedback controller* (PD controller).

$$k(t) = K_p e(t) + K_d \frac{de(t)}{dt} \quad (6.2)$$

Now, not only will the control effort $k(t)$ respond to error, but it will also respond to changes in the error. When your roommate opens the window and lets that frigid January air into your room, your controller will recognize the high error derivative and immediately kick on the space heater, keeping your room nice and toasty!

6.7 PID Feedback Control

Our PD controller is quite good at this point. It can respond to both error and change in error, which makes it very robust against a variety of system changes (opening a window, temperature outside changes, and so on). There's one small issue. Let's say that your roommate opens the window (seriously? this time of year?) and the temperature in your room quickly drops below your setpoint. Your space heater will respond quickly and stabilize the temperature in your room, but it might stabilize to one or two degrees below the setpoint. The proportional term $K_p e(t)$ will slowly work towards reducing this, but it could take a very long time. The closer you get to the setpoint, the less the proportional term affects the result. The derivative term $K_d de(t)/dt$ is even more useless, because if the temperature's stable, then the derivative is zero and it doesn't contribute at all. What you want is some way to adjust based on *long-term persistence of error*. We can keep track of the error over time using an integral, so let's add a new term with constant K_i . Equation (6.3) describes our new *proportional-integral-derivative feedback controller* (PID controller).

$$k(t) = K_p e(t) + K_i \int_0^t e(t') dt' + K_d \frac{de(t)}{dt} \quad (6.3)$$

PID control is the industry standard in feedback *control policy* because of its ability to respond to plant changes quickly and in a highly robust manner.

Lecture 7

Kinematics & Path Planning

For your robot to move itself across the game field, it will need to be able to create and follow path plans. In this lecture, we'll explore the kinematics of Kitbot and use our newfound understanding to create robust path plans! Of course, if you choose to use a different robot geometry, we invite you to follow along to derive a model accurate to your robot's design.

7.1 Forward Kinematics

To move around, your robot will need incorporated into its controller a model of its own geometry. To make an analogy to your everyday experience, in order to walk around, your brain needs to understand how long your legs are and how your body weight is distributed. In the same way, your robot should “understand” the diameter and spacing of its wheels in order to move in a coordinated fashion. Let's start by defining the variables of concern in the problem. We'll use c to denote the wheelbase of your robot and r to denote the radius of your robot's wheels. We'll also use v and θ to denote the forward velocity and rotation rate of the robot, respectively. Importantly, $\theta = 0$ will correspond to the robot facing in the direction of increasing x . Finally, we will use Ω_R and Ω_L to denote the rotation rates of the right and left wheels, respectively. Consider a small amount of time Δt . The right wheel will rotate by the angle $\Omega_R \Delta t$ and the left wheel will rotate by the angle $\Omega_L \Delta t$. This will cause the right wheel's contact point to advance forward by $r\Omega_R \Delta t$ and the left wheel's contact point to advance forward by $r\Omega_L \Delta t$. Take a look at Figure 7.1 for a visual representation of this.

As you can see in the diagram, we can construct a triangle containing the angle $\dot{\theta} \Delta t$, which is the angle by which the robot has rotated. We can also see that the center of the robot has travelled forward by $v \Delta t$.

$$v \Delta t = \frac{r}{2} \Delta t (\Omega_R + \Omega_L) \Rightarrow v = \frac{r}{2} (\Omega_R + \Omega_L) \quad (7.1)$$

$$\tan(\dot{\theta} \Delta t) = \frac{r}{c} \Delta t (\Omega_R - \Omega_L) \Rightarrow \dot{\theta} = \frac{1}{\Delta t} \arctan\left(\frac{r}{c} \Delta t (\Omega_R - \Omega_L)\right) \quad (7.2)$$

We may assume that, as $\Delta t \rightarrow 0$, the arctangent function linearizes around $x = 0$ and the substitution $\arctan x \approx x$ becomes increasingly accurate.

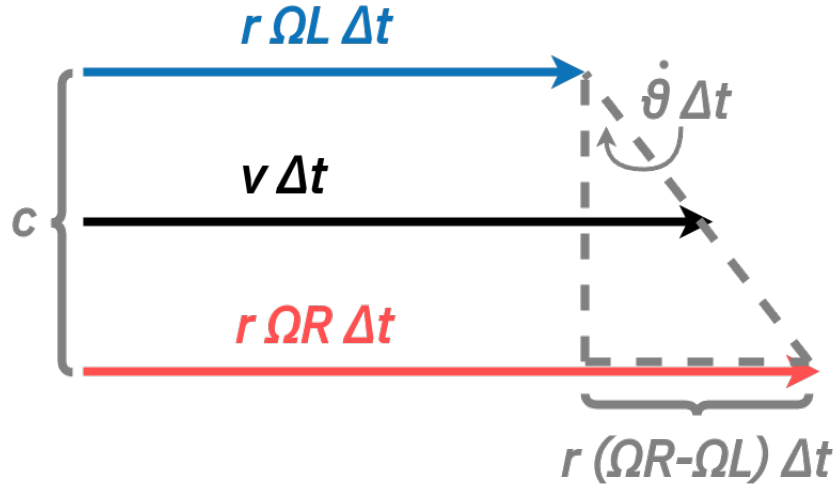


Figure 7.1: Kinematic Model for Kitbot

$$\tan(\dot{\theta}\Delta t) = \frac{r}{c}\Delta t(\Omega_R - \Omega_L) \Rightarrow \dot{\theta} = \frac{r}{c}(\Omega_R - \Omega_L) \quad (7.3)$$

This yields for us a system of equations describing the *forward kinematics* of your robot. The system takes two inputs, Ω_R and Ω_L , and returns two outputs, v and $\dot{\theta}$. We can use matrix form to describe this system, like so. We're using this form specifically because you'll often see complex systems described in this manner, and it opens up the door to using the powerful tools of linear algebra. All you need to understand for now, though, is that the output is on the left-hand side and the input is on the right-hand side. We can use this to calculate the forward and rotational velocity of your robot from given wheel rotation rates.

$$\begin{pmatrix} v \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} r/2 & r/2 \\ r/c & -r/c \end{pmatrix} \begin{pmatrix} \Omega_R \\ \Omega_L \end{pmatrix} \quad (7.4)$$

7.2 Inverse Kinematics

This is great and all, but it's not very useful to us. When our robot creates its path plans, it won't know what wheel rotation rates to use, but rather it will know the velocity it needs to get to a given location on the game field. Essentially, we need to create the inverse system as the one we've designed... or in other words, we need to understand the *inverse kinematics*. This is pretty easy using the tools of linear algebra. All we need to do is invert the matrix in (7.4).

$$\begin{pmatrix} \Omega_R \\ \Omega_L \end{pmatrix} = \begin{pmatrix} r/2 & r/2 \\ r/c & -r/c \end{pmatrix}^{-1} \begin{pmatrix} v \\ \dot{\theta} \end{pmatrix} \quad (7.5)$$

We'll use a computer algebra system to invert this matrix. You can also write the system of equations and solve it by elimination, if you desire. You'll get the same answer either way!

$$\begin{pmatrix} \Omega_R \\ \Omega_L \end{pmatrix} = \begin{pmatrix} 1/r & c/2r \\ 1/r & -c/2r \end{pmatrix} \begin{pmatrix} v \\ \dot{\theta} \end{pmatrix} \quad (7.6)$$

Much better! Now, given a desired forward and angular velocity, we can solve for the wheel rotation rates we need.

7.3 Path Plans

We now have a model for our robot's kinematics, and we understand it in both the forward and reverse directions. Now, we want to construct a *path plan* that will enable us to drive from Point A to Point B in a smooth, controlled manner. For this purpose, we will define a path plan as a function $P : \mathbb{R} \rightarrow \mathbb{R}^2$ that maps a time t to a pair of real numbers $(v, \dot{\theta})$. In English, a path plan is a correspondence between every point in time and the desired motion of the robot at that point in time. A very simple path plan might be the one below.

$$P(t) = \begin{cases} (0, 0) & t < 0 \\ (1, 0) & t \geq 0 \end{cases} \quad (7.7)$$

What does this path plan do? Well, for $t < 0$, the robot doesn't move at all. At time $t = 0$, the robot begins to move forward at velocity 1 m/s but does not rotate, and it continues to do so forever. Not very useful, is it? Let's try another one.

$$P(t) = \begin{cases} (0, 0) & t < 0 \\ (0, 1) & t \geq 0 \end{cases} \quad (7.8)$$

Again, for $t < 0$, the robot doesn't move at all. Then, at time $t = 0$, the robot begins to spin counterclockwise in place at rate 1 rad/s and continues to do so forever. Again, not all that useful, is it? But it does provide us with good insight into how the path plan influences the motion of the robot. Let's try a more complicated one.

$$P(t) = \begin{cases} (0, 0) & t < 0 \\ (1, 1) & t \geq 0 \end{cases} \quad (7.9)$$

This path plan is a little more interesting. At $t = 0$, the robot begins moving forward at 1 m/s *and* counterclockwise at 1 rad/s. What kind of trajectory does this path plan create?

7.4 Trajectories

Before we can actually use this concept of a path plan, we're missing one critical piece. How do we convert between v and $\dot{\theta}$ and the actual position of the robot on the field? For this, we'll define a *trajectory* as another function $T : \mathbb{R} \rightarrow \mathbb{R}^3$ mapping a time t to a triplet of real numbers representing a position and orientation (x, y, θ) on the game field. Let's represent this in Figure 7.2, for a small time step Δt .

From this diagram, we can write the following relationships.

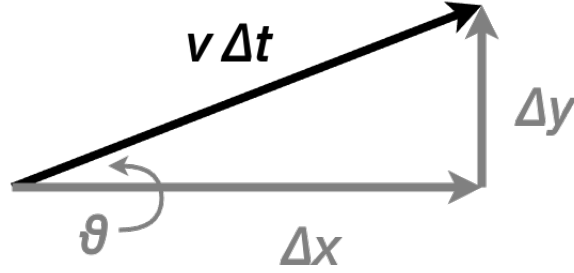


Figure 7.2: Trajectory Model for Kitbot

$$\Delta x = v \Delta t \cos \theta \quad (7.10)$$

$$\Delta y = v \Delta t \sin \theta \quad (7.11)$$

We can also express the change in the angle θ in terms of $\dot{\theta}$.

$$\Delta \theta = \dot{\theta} \Delta t \quad (7.12)$$

From this, we can deduce a system of differential equations that describe the position and orientation of the robot from its velocities.

$$\dot{x} = v \cos \theta \quad (7.13)$$

$$\dot{y} = v \sin \theta \quad (7.14)$$

$$\dot{\theta} = \dot{\theta} \quad (7.15)$$

This last equation may seem a bit trivial, but it is important because $\dot{\theta}$ is both an *output* of the forward kinematics model and an *input* of the trajectory system! We will again rewrite this system in terms of a matrix, with the output on the left and the input on the right.

$$\boxed{\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} v \\ \dot{\theta} \end{pmatrix}} \quad (7.16)$$

Excellent! Now, given a forward and rotational velocity, we can solve for the movement of the robot in space. Notice that we don't have an explicit equation for the robot's trajectory; this is difficult to calculate in its general form. However, we will use this system later to solve for specific robot trajectories.

7.5 Solving For Trajectories

Remember that path plan we created earlier?

$$P(t) = \begin{cases} (0, 0) & t < 0 \\ (1, 1) & t \geq 0 \end{cases} \quad (7.17)$$

Now, using everything in our toolbox, we can write a system of differential equations for x , y , and θ given this path plan.

$$\dot{x} = \cos \theta \quad (7.18)$$

$$\dot{y} = \sin \theta \quad (7.19)$$

$$\dot{\theta} = 1 \quad (7.20)$$

Assuming the initial conditions $x(0) = 0$, $y(0) = 0$, and $\theta(0) = 0$, we can solve for the full trajectory.

$$x(t) = \sin t \quad (7.21)$$

$$y(t) = 1 - \cos t \quad (7.22)$$

$$\theta(t) = t \quad (7.23)$$

Figure 7.3 shows a plot of this trajectory. The robot drives in a circle! Clearly, it might be very difficult to imagine what this looks like if we didn't have a rigorous way of defining the path plan and computing its associated trajectory. Don't worry if you don't yet feel comfortable with every step of how we got here... what's important to understand is that we have a robust way of getting from a *path plan* (velocity profile over time) to its associated *trajectory* (position and orientation over time) given some *initial conditions* (where the robot started). We can now combine this work with our robot's kinematics to understand the relationship between wheel rotation rates and robot trajectories.

7.6 Creating Path Plans

Let's suppose our robot is sitting at $(0, 0)$ and we'd like to drive to some destination point $(x_f, 0)$. Let's additionally suppose that the robot starts at the angle $\theta = 0$ (facing to the right). How can we devise a trajectory and an associated path plan that satisfy these constraints in a practical manner? And then, more importantly, how can we come up with wheel rotation rates that will get us where we need to go?

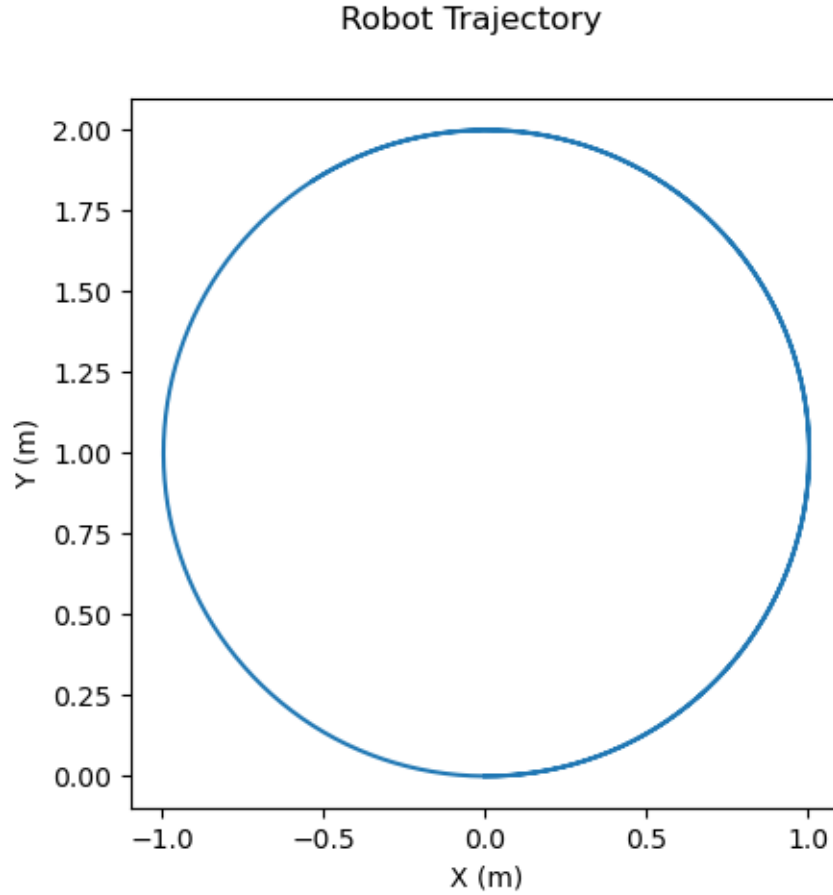


Figure 7.3: Robot Trajectory

7.6.1 Constant Velocity Path Plan

The simplest path plan we could conceive of is a *constant velocity path plan*. We first set our maximum velocity v_{max} , which might be determined either by the physical limitations of our robot, a safety limitation, or other considerations. We can then write our path plan P as in (7.24).

$$P(t) = \begin{cases} (v_{max}, 0) & 0 \leq t < x_f/v_{max} \\ (0, 0) & \text{otherwise} \end{cases} \quad (7.24)$$

As mentioned before, it can be hard to visualize equations, so let's plot this path plan in Figure 7.4, with the robot's associated position along the x -direction and its acceleration. Here, our robot is traveling 10 meters with a maximum velocity of 4 m/s.

This is a decent path plan, but it certainly has issues. The biggest among these is that we're expecting the robot to accelerate instantaneously! Obviously, infinite acceleration isn't possible, and so in reality, it's not actually possible to follow this plan... there will be a small delay, however short, as the robot speeds up to v_{max} and then slows down at the end of its

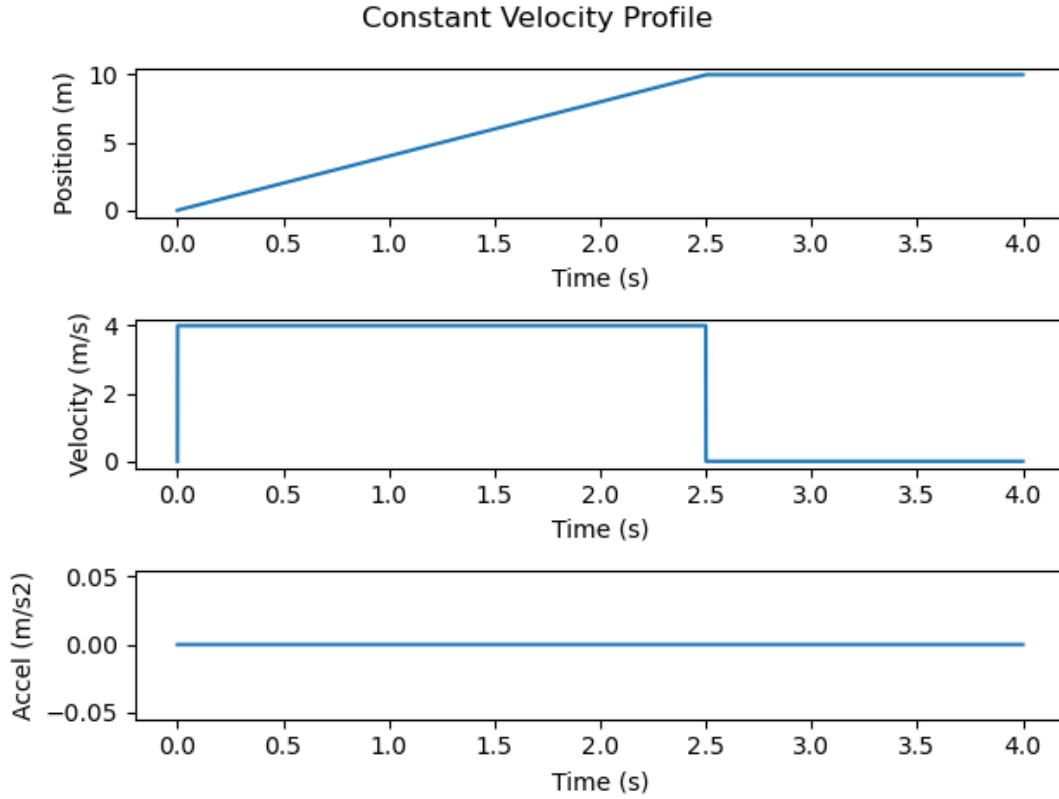


Figure 7.4: Constant Velocity Path Plan

journey, and this delay will cause us to deviate from our path plan. That's not good. Can we do better?

7.6.2 Triangular Path Plan

Infinite acceleration is obviously impossible.^[citation needed] So let's determine the maximum acceleration of our robot and denote it as a_{max} . Now, we can make our robot accelerate until it's halfway there, then decelerate until it arrives at its destination. The path plan will look a bit more complicated, but we can still write it down as in (7.25).

$$P(t) = \begin{cases} (a_{max}t, 0) & 0 \leq t < \sqrt{x_f/a_{max}} \\ (2\sqrt{x_f a_{max}} - a_{max}t, 0) & \sqrt{x_f/a_{max}} \leq t < 2\sqrt{x_f/a_{max}} \\ (0, 0) & \text{otherwise} \end{cases} \quad (7.25)$$

That's a mess! Let's plot this again like before. Figure 7.5 shows the position, velocity, and acceleration associated with this path plan, assuming a maximum acceleration of 5 m/s². This is much better! Now, we no longer are assuming that the robot can accelerate instantaneously, so our path plan is much more realistic and we're less likely to introduce error into our robot's trajectory. We call this a *triangular path plan* because the velocity

profile looks like a triangle; it increases and then decreases.

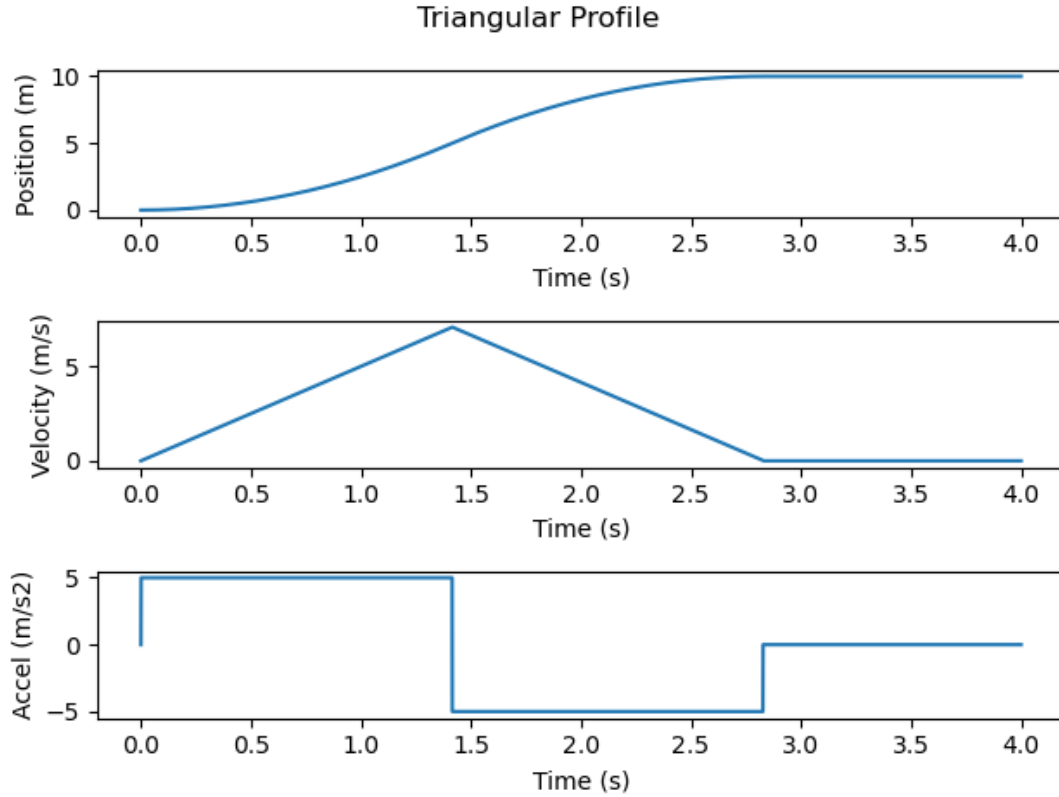


Figure 7.5: Triangular Path Plan

7.6.3 Trapezoidal Path Plan

There's just one issue here. Remember our maximum velocity constraint v_{max} ? Look again at Figure 7.5. We're meeting our maximum acceleration constraint, but now we've violated our maximum velocity constraint of 5 m/s. How can we construct a path plan in which we meet both constraints? To fix the issue, we'll accelerate at a_{max} until we reach maximum velocity, then wait until we're almost at our destination (but not quite). Then, we can start decelerating at maximum acceleration until we stop right on target. The path plan will look even more complicated than before, but we can still write it analytically as in (7.26).

$$P(t) = \begin{cases} (a_{max}t, 0) & 0 \leq t < t_1 \\ (v_{max}, 0) & t_1 \leq t < t_2 \\ (v_{max} - a_{max}(t - t_2), 0) & t_2 \leq t < t_3 \\ (0, 0) & \text{otherwise} \end{cases} \quad (7.26)$$

The constants t_1 , t_2 , and t_3 above are subject to the below constraints.

$$t_1 = \frac{v_{max}}{a_{max}} \quad (7.27)$$

$$t_2 = t_1 + \frac{x_f - a_{max}t^2}{v_{max}} \quad (7.28)$$

$$t_3 = t_2 + t_1 \quad (7.29)$$

What a mess! Let's plot the position, velocity, and acceleration associated to this path plan again in Figure 7.6, to build our visual intuition. Perfect! Now, we obey our maximum acceleration constraint of 5 m/s² *and* our maximum velocity constraint of 4 m/s. Notice how the velocity profile is shaped like a trapezoid? This is a *trapezoidal path plan*, a commonly used path plan used in the field of robotics.

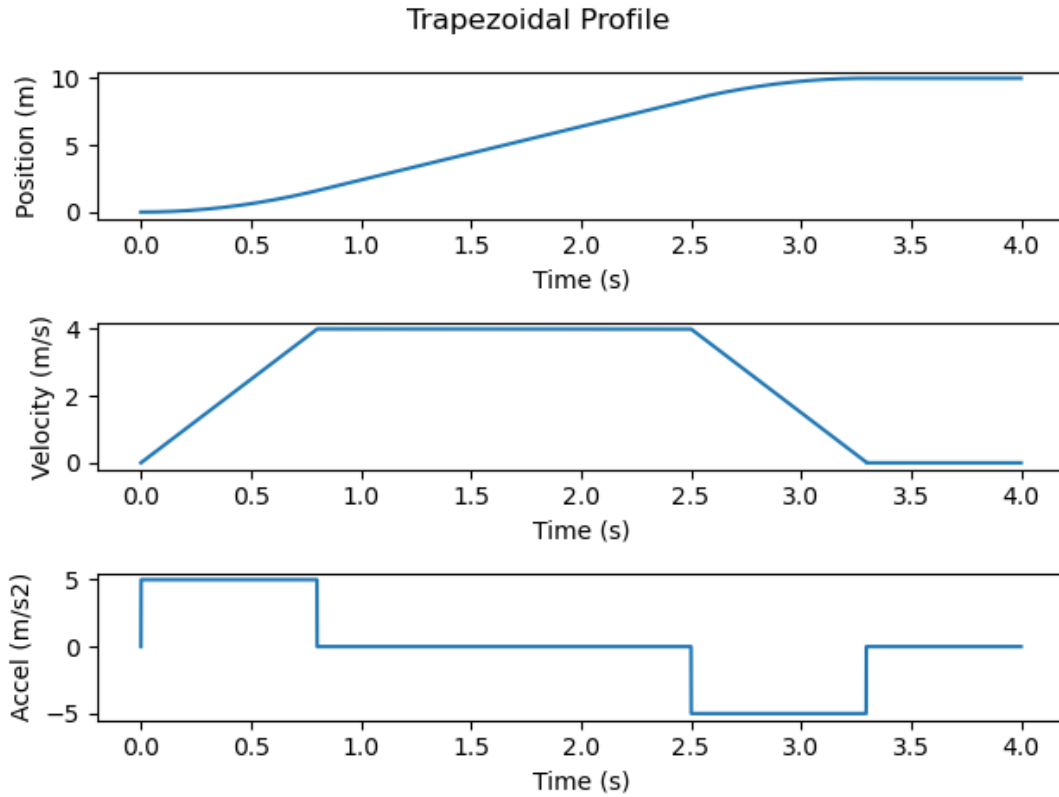


Figure 7.6: Trapezoidal Path Plan

7.6.4 Other Path Plans

Trapezoidal path plans are quite effective, and we encourage you to use them while planning your robot's journey across the competition field. That said, there are additional considerations that can lead us to even more complicated path plans. For example, quick accelerations

and decelerations can cause slipping or vibration, which are typically considered undesirable. To correct for that, some robotics engineers will use *S-curve path plans*, in which the robot's acceleration itself has a trapezoidal profile. This typically requires accounting for the third-order time derivative of position. If velocity is the first-order time derivative of position, and acceleration is the second-order derivative, the third-order derivative is typically referred to as *jerk*. For the purposes of MASLAB, we consider the complexities of S-curves to exceed their benefits, though you are welcome to use them if you desire.

Lecture 8

Computer Vision

Autonomous robots, by their very definition, have to operate without human control. This means that your robot will have to see with its own eyes on the competition field. Advances in computation at the Institute and beyond in the last 70 years have given birth to the field of *computer vision*. Developments in computer vision allow computational systems to reason about visual information as collected from a digital camera. Here, we'll explore the foundations of computer vision, as well as how we can apply the technologies and techniques available to us to allow your robot to see for itself.

8.1 Color Spaces

To understand the way that computers reason about images, we need to understand the way that computers reason about color in a formal way. Computers (and engineers) reason about colors formally using the concept of *color space*. Color spaces are a rigorous way to organize and think about colors.

8.1.1 RGB Color Space

The most common and most familiar color space used in computing is RGB color space. A color c in RGB color space is represented as a triplet of real numbers (r, g, b) subject to $0 \leq r, g, b \leq 1$. The symbols (r, g, b) represent the amount of red, green, and blue contained in the color c . RGB color space is so widespread because of digital displays; displays create colors by combining red, green, and blue light at different intensities in each pixel. Digital cameras work using the same principle; they detect and record the intensities of red, green, and blue light at each pixel to construct an image.

RGB color space is the most natural color space to work with, but it's not particularly useful for computer vision. The biggest issue with RGB color space is that it's pretty difficult to understand "what color" a given color is. You and I don't think of colors as triplets of three numbers (maybe you do, but I certainly don't). We like to think of colors as "bright red" or "cornflower blue". But how easy is it to tell that $(0.859, 0.635, 0.156)$ is a nice pumpkin orange? RGB color space doesn't communicate any information about a color's *hue*. Hue is the "orange-ness" of orange and the "blue-ness" of blue. Maybe there's another color space

that is more suited to thinking about hues.

8.1.2 Defining Hue

Given an arbitrary color (r, g, b) , let's come up with a way of numerically representing the hue of this color. Remember color wheels from elementary school art class? Maybe we can describe hue h as an angle along this circle, subject to $0 \leq h < 2\pi$. We can start defining pure red, pure green, and pure blue as equally spaced along this color wheel as $h = 0$ (0°), $h = 2\pi/3$ (120°), and $h = 4\pi/3$ (240°). Figure 8.1 shows the color wheel with the primary colors (red, green, and blue) marked at their respective positions. Notice that the secondary colors (yellow, cyan, and magenta) have also been added.

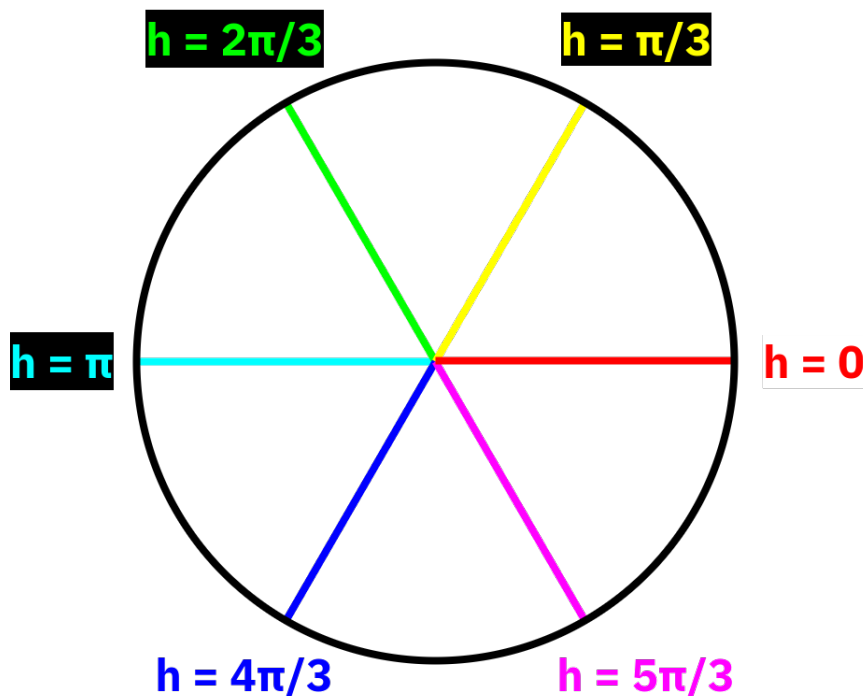


Figure 8.1: Hues on the Color Wheel

How do we interpolate between these three colors? Let's simplify the problem a bit by working with colors such that $\max(r, g, b) = r$. These are colors that exist in the sector of the color wheel between magenta and yellow. There's a lot of colors in this space (actually, an uncountably infinite number of colors), and we need to be able to organize them in a meaningful manner. Maybe we should describe the hue of a color c in this sector as $\pi/3 \cdot (g - b)$? This way, if the color is closer to green than blue, then the hue is positive, but if the color is closer to blue than green, then the hue is negative. Pure yellow is $(1, 1, 0)$, which converts correctly to the hue $h = \pi/3$. But does this tell the whole story?

8.1.3 Chroma

Not quite. Take a look at two colors $c_1 = (0.5, 0.4, 0.25)$ and $c_2 = (1, 0.8, 0.5)$ below in Figure 8.2. You might describe c_1 as coffee brown and c_2 as creamsicle orange. Sure, they're different colors... but are they different *hues*? Imagine you were a painter. Do you see how you could make c_2 by mixing c_1 with white, or how you could make c_1 by mixing c_2 with black? White and black themselves have no hue, so mixing a color with either of these shouldn't change its hue. From this argument, it's reasonable for us to conclude that c_1 and c_2 have the same hue.

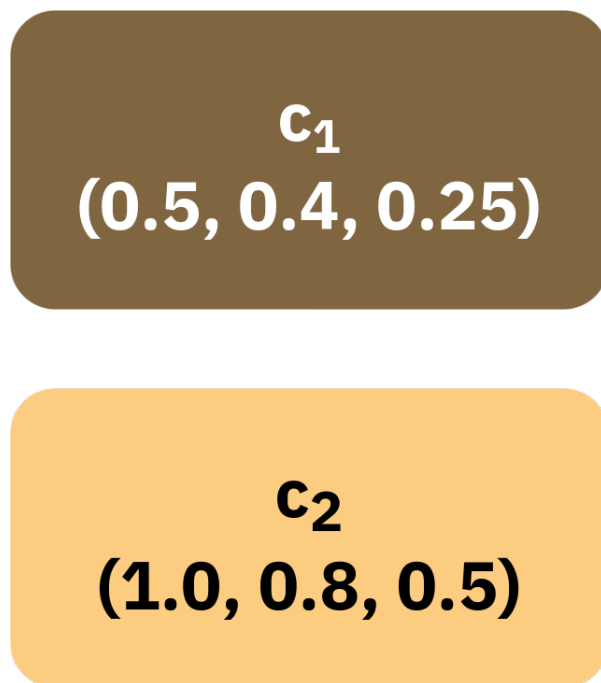


Figure 8.2: Same Hue, Different Chromas

Under our current model for hue, these colors work out to be $h \approx 0.157$ and $h \approx 0.314$ respectively, which disagrees with our above conclusion. We want c_1 and c_2 to have the same hue but different levels of *chroma*, which is a measurement of a color's vividness. Color c_2 is much more vivid than c_1 (has more chroma), but they're still the "same color" on a color wheel. In RGB color space, shades of gray are represented with three equal values like $(0.5, 0.5, 0.5)$, and gray should have zero chroma. Let's define $c_{max} = \max(r, g, b)$ and $c_{min} = \min(r, g, b)$. When $c_{min} = c_{max}$, then we're working with white, black, or gray, and we consider its chroma to be zero. When c_{min} and c_{max} are very far apart, we're working with a high chroma (very intense) shade of a color. Therefore, we'll define chroma as $\Delta := c_{max} - c_{min}$, which satisfies both of these requirements. Under this definition, the chroma for c_1 and c_2 works out to be $\Delta = 0.25$ and $\Delta = 0.5$ respectively, which feels accurate... c_2 is a more vivid color than c_1 , so it should have a higher chroma value.

8.1.4 Fixing Hue

Now that we've characterized the difference between c_1 and c_2 accurately, we can go back and fix our definition of hue. Notice how both the hue and the chroma of c_2 are double those of c_1 ? Let's correct our definition of hue to be $h = \pi/3 \cdot (g - b)/\Delta$. Now that we're dividing by chroma, the hues of both c_1 and c_2 are equal at $h = 0.628$. This is much better. However, this definition only applies to colors that are between yellow and magenta, so let's extend it to the entire color wheel using a piecewise definition as in (8.1).

$$h := \frac{\pi}{3} \begin{cases} (g - b)/\Delta & c_{max} = r \\ 2 + (b - r)/\Delta & c_{max} = g \\ 4 + (r - g)/\Delta & c_{max} = b \\ 0 & \Delta = 0 \end{cases} \quad (8.1)$$

Notice that we've shifted the hue of colors close to green and colors close to blue in order to complete the circle. Also, in order to avoid division by zero, we've included an additional case to account for colors with zero chroma (black, white, and gray), which are considered to have zero hue.

8.1.5 HSV Color Space

This definition of hue is great. We can now organize colors into “reds”, “greens”, or “oranges” by their hue values without having to be confused by RGB color space. We can actually extend this model to create a whole new color space. Firstly, we define *value* as $v = c_{max}$ and *saturation* as $s = \Delta/v$. Value tells us the “brightness” of a color, whereas saturation tells us the “intensity” of a color, very similarly to chroma. Hue, saturation, and value form the basis for *HSV color space*, which defines colors as triplets of three numbers (h, s, v) , subject to $0 \leq h < 2\pi$ and $0 \leq s, v \leq 1$. We encourage you to use HSV color space for computer vision in this course, as it's far more robust against changes in lighting conditions and other environmental factors than RGB color space.

8.2 Image Segmentation

Now that we have a rigorous way of understanding color, how can we apply this model to computer vision? One common task in computer vision is *image segmentation*, which involves partitioning a digital image into “segments”. This process is commonly used in object detection, which is our exact goal. Let's suppose we're trying to detect a red block placed somewhere in front of our robot. Our existing model of HSV color space as well as the process of image segmentation makes this seemingly difficult task very simple. We can write a simple program consisting of the following steps.

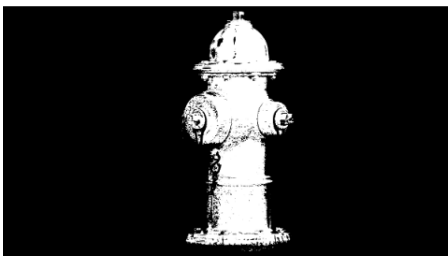
1. Determine the hue value of the object to be detected, h .
2. Select a subset S of HSV color space, such that every color $c \in S$ is “similar in hue to h ”. For example, we might consider a hue to be “similar in hue to h ” to be between $h - 0.1$ and $h + 0.1$.

3. Receive a digital image from the camera.
4. Convert the image data from RGB color space to HSV color space.
5. Classify each pixel as $c \in S$ (“similar in hue to h ”) or $c \notin S$ (“not similar in hue to h ”).
6. Identify the largest continuous segment of pixels that are subject to $c \in S$. We can be pretty confident that this is the object we’re looking for!

Maybe that wasn’t so simple... so let’s take a look at Figure 8.3. We’re trying to use image segmentation to isolate a red fire hydrant in a field of green grass. We’ve converted the image into HSV color space and, in the second frame, highlighted in white each pixel that satisfies $5\pi/3 \leq h < 2\pi$, $0.2 \leq s \leq 1$, and $0.2 \leq v \leq 1$. The cyan box in the third frame outlines the largest continuous segment of highlighted pixels. Very simple... and very effective! Notice that, most importantly, our use of HSV color space allows us to generalize this to any hue without having to worry about the shadows cast by the shape of the hydrant.



Original image



Selected pixels



Identified segment

Figure 8.3: Image Segmentation on a Fire Hydrant

8.3 Projective Geometry

Using HSV color space and the image segmentation algorithm we demonstrated previously, we can now identify objects in images by their color. The next step in computer vision is determining their position relative to your robot, so your robot can begin creating a map of the field around it. But there's just one issue. Your robot is driving around in our 3-dimensional world, and the information coming from your camera is 2-dimensional, so we are fundamentally missing information. An understanding of *projective geometry* will help us significantly here. Projective geometry gives us a new perspective into the field of geometry... it allows us to understand what changes (and what doesn't) when we *project* 3-dimensional objects onto a 2-dimensional image. We can begin to develop an understanding of projective geometry by thinking about a projector! Take a look at Figure 8.4.

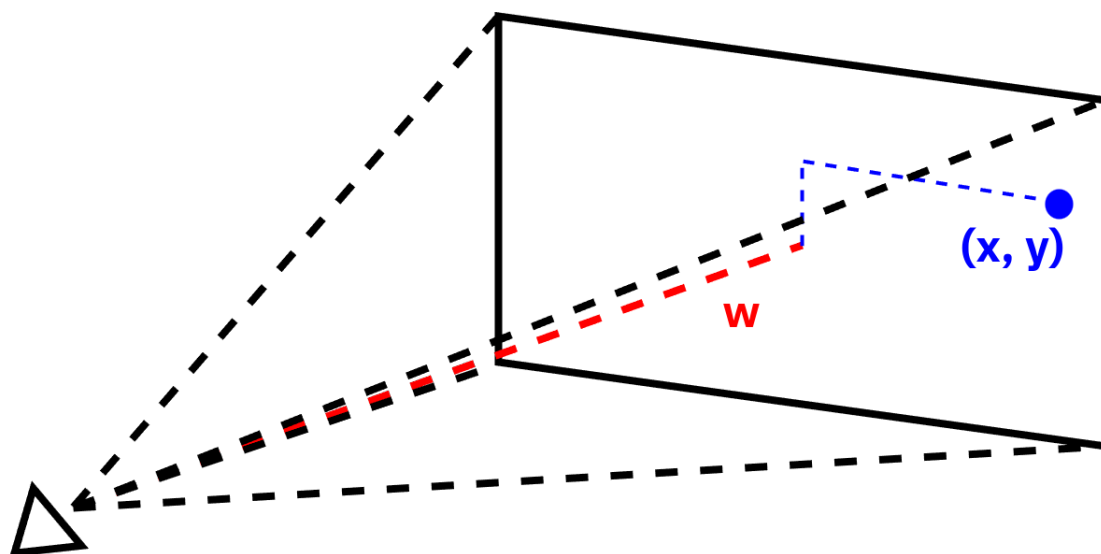


Figure 8.4: A Projector and a Screen

Notice that the point P exists at coordinate (x, y) on the 2-dimensional screen. We also denote the distance between the projector and the screen as w , for reasons that will become clear later. What happens to P when we move the screen to be a distance of $2w$ from the projector? Figure 8.5 demonstrates this new setup.

Take a look at the location of P . It's moved on the screen to $(2x, 2y)$. In a certain sense, the point P hasn't moved at all. But in another sense, it *has* moved because the projector moved! How can we encapsulate this distinction in a rigorous way?

8.3.1 Homogeneous Coordinates

Here's an idea. Instead of describing the position of P with two real numbers, like (x, y) , let's describe it as (x, y, w) . We can then think about the position of P *on the screen* as some sort of coordinate normalized by w , like $(x/w, y/w, 1)$. Now, when we move the screen closer or further from the projector, w changes, but so do x and y in equal proportion, and their ratios

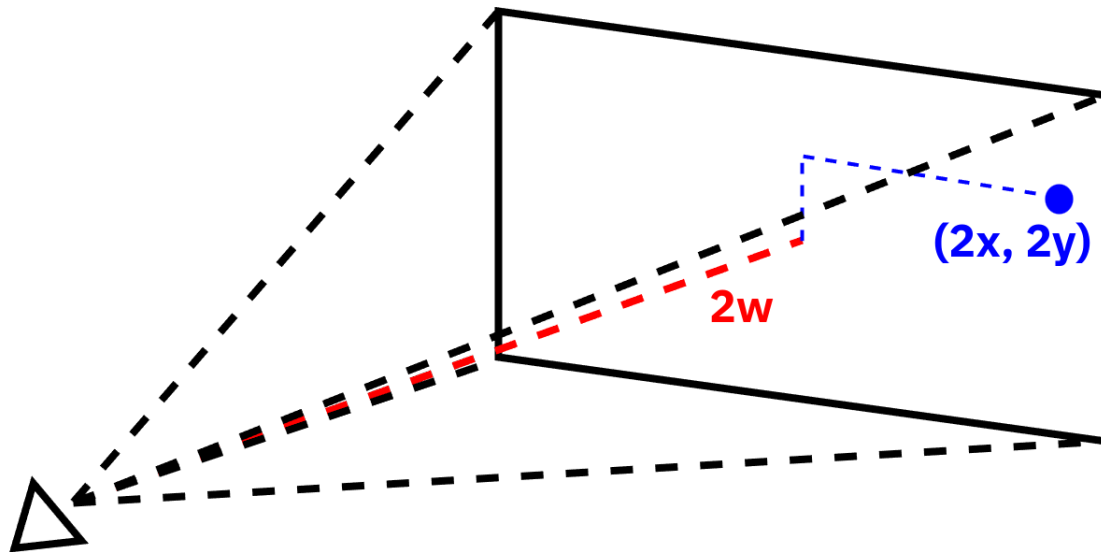


Figure 8.5: Moving the Screen

x/w and y/w stay the same! This feels kind of weird at first, but this representation offers an enormous benefit: it has perspective “baked in”! We call this coordinate system *homogeneous coordinates*. Computer vision and computer graphics engineers use homogeneous coordinates to understand and work with projective geometry.

Let’s step back into the 3-dimensional world and think about a new example. Consider a point R that exists in 3-dimensional space at location (x, y, z) in Cartesian coordinates. We can convert this point into a point in homogeneous (projective) coordinates by writing it as $(x, y, z, 1)$. Whenever $w = 1$, we consider the point to be *normalized*, and the x , y , and z values are representative of its position in Euclidean space. Let’s now consider a second point, Q , located at $(2x, 2y, 2z)$ in Cartesian coordinates and $(2x, 2y, 2z, 1)$ in homogeneous coordinates. Now suppose we are standing at the origin and looking in the positive x -direction. We should see these points as one behind the other, because we can draw a line from the origin through P and Q . How can we check this mathematically? We will project these coordinates into *projective space* using the *perspective projective matrix* in (8.2). This particular projective matrix represents the perspective of looking in the positive x -direction. The vector $\vec{p}$ is a position in 3-dimensional projective space, and the vector $\vec{x}$ is a homogeneous coordinate.

$$\vec{p} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \vec{x} \quad (8.2)$$

Below, we project P and Q into projective space and *normalize* the resultant vector.

$$P : \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ x \end{pmatrix} \rightarrow \vec{p} = \begin{pmatrix} 1 \\ y/x \\ z/x \\ 1 \end{pmatrix} \quad (8.3)$$

$$Q : \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 2x \\ 2y \\ 2z \\ 1 \end{pmatrix} = \begin{pmatrix} 2x \\ 2y \\ 2z \\ 2x \end{pmatrix} \rightarrow \vec{q} = \begin{pmatrix} 1 \\ y/x \\ z/x \\ 1 \end{pmatrix} \quad (8.4)$$

Notice something interesting. Vectors $\vec{p}$ and $\vec{q}$ are mapped to the same point in projective space! What this means is that, in our perspective as we look in the positive x -direction, P and Q are in the same place, just as we expected.

8.3.2 Distance Estimation

How can we use homogeneous coordinates to estimate the distance of a block? Let's say that, using the image segmentation technique we developed above, we've identified a green block on the playing field ahead, and we want to estimate its distance, which we'll denote d . First, we'll establish our coordinate system. Suppose this time, we are looking in the negative z -direction, so that the positive x -direction points to our right and the positive y -direction points upward. The corresponding projective matrix in this case is going to be

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix}. \quad (8.5)$$

Suppose also that the block is centered at $(0, 0, -d)$ in our field of view and is a cube with side length L . The top right front corner of the cube, which we'll call R , is therefore located at

$$\begin{pmatrix} L/2 \\ L/2 \\ -d + L/2 \\ 1 \end{pmatrix} \quad (8.6)$$

in homogeneous coordinates. Take a look at Figure 8.6 to get a general idea of the setup we're looking at. Let's apply our projective matrix and normalize the result to apply perspective to the point R . Equation (8.7) shows R and its equivalent perspective projection.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} L/2 \\ L/2 \\ -d + L/2 \\ 1 \end{pmatrix} = \begin{pmatrix} L/2 \\ L/2 \\ -d + L/2 \\ d - L/2 \end{pmatrix} \rightarrow \begin{pmatrix} 2L/(2d - L) \\ 2L/(2d - L) \\ -1 \\ 1 \end{pmatrix} \quad (8.7)$$

We're going to discard the z -coordinate here, because that's what a camera does... it only captures 2-dimensional information! So it seems that the point R is projected to a point

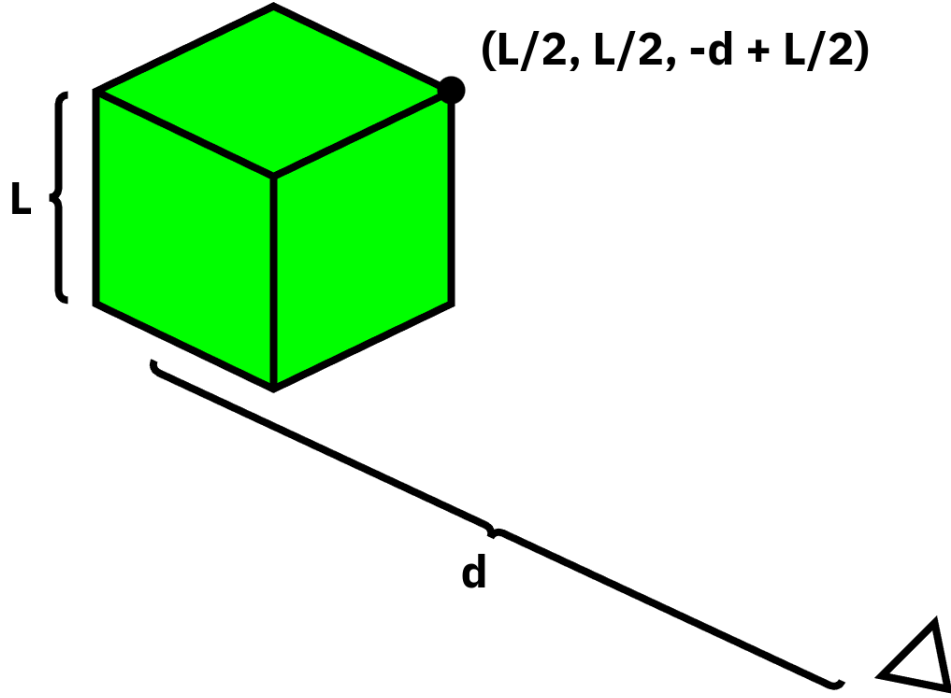


Figure 8.6: Projecting a Cube

$\langle 2L/(2d-L), 2L/(2d-L) \rangle$ on the camera's projective plane. How does this translate to pixel location in the resultant image? Firstly, we assume that the camera sensor has dimensions ℓ by h , and that it's located at a focal length f behind the origin. Let's project the bottom left corner of the camera's sensor, denoted S , located at $(-\ell/2, -h/2, f)$ in Cartesian space, onto the projective plane. Figure 8.7 should help you visualize what's going on here. We're assuming a *pinhole camera* model, which means that the camera flips the image vertically and horizontally before the computer corrects the image. Equation (8.8) shows the point S and its equivalent perspective projection.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} -\ell/2 \\ -h/2 \\ f \\ 1 \end{pmatrix} = \begin{pmatrix} -\ell/2 \\ -h/2 \\ f \\ -f \end{pmatrix} \rightarrow \begin{pmatrix} \ell/2f \\ h/2f \\ -1 \\ 1 \end{pmatrix} \quad (8.8)$$

Again, we discard the z -coordinate because our camera only captures 2-dimensional data. Now, there's just one more step. We'll assume that the image has pixel dimensions (W, H) and calculate where on the image R ends up. The point S ends up at $(W/2, H/2)$ relative to the center of the image, so we can determine the position of R on the image (x_{px}, y_{px}) relative to this. It's very important to note that (x_{px}, y_{px}) is relative to the *center* of the image.

$$x_{px} = \frac{2L}{2d-L} \cdot \frac{2f}{\ell} \cdot W \quad (8.9)$$

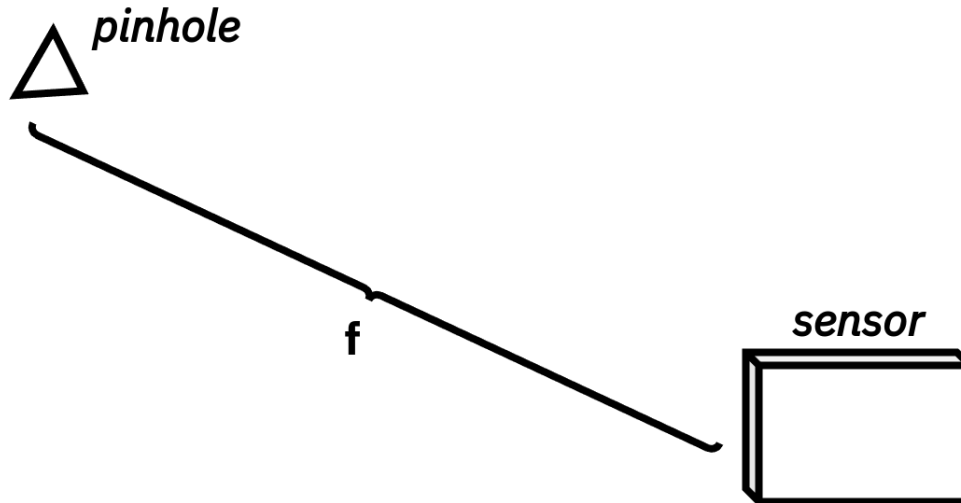


Figure 8.7: Projecting the Camera Sensor

$$y_{px} = \frac{2L}{2d - L} \cdot \frac{2f}{h} \cdot H \quad (8.10)$$

Oh my goodness, that was a lot! Finally, we've worked out the pixel position of R on the image. In practice, we'll know every variable here except d , so we can solve these equations for d based on the top right corner of the block on the image (x_{px}, y_{px}); the side length of the block L ; the width W and height H of the image in pixels; the width ℓ and height h of the camera's sensor; and the camera's focal length f . Equation (8.11) gives the expression for d based on these known values.

$$d = L \left(1 + \frac{W}{x_{px}} \cdot \frac{2f}{\ell} \right) = L \left(1 + \frac{H}{y_{px}} \cdot \frac{2f}{h} \right) \quad (8.11)$$

Lecture 9

Machine Learning

Begins here.